

Raccolta di domande per lo scritto e l'orale di Sistemi Operativi

FILIPPO GIANNECCHINI

NOTE DI CHI HA CARICATO IL DOCUMENTO (GIUSEPPE VAGLICA):

RIPORTO IL MESSAGGIO MANDATO DALL'AUTORE PER COMPLETEZZA

“CI HO SPESO TROPPO TEMPO PER LASCIARLO MORIRE COSÌ SUL MIO COMPUTER. QUESTO FILE CONTIENE TUTTE LE DOMANDE CHE SONO RIUSCITO A RECUPERARE DA VARIE FONTI SIA DELLO SCRITTO CHE DELL' ORALE. AH LA ROBA CHE VEDETE IN ROSSO SONO SEMPLICEMNTE DOMANDE CHE AVEVO SBAGLIATO MENTRE RIPASSAVO, NULLA DA TENER DI CONTO

NON ASSICURO CHE LE RISPOSTE SIANO TUTTE CORRETTE E/O COMPLETE AL 100%, USATELO A VOSTRO RISCHIO E PERICOLO.”

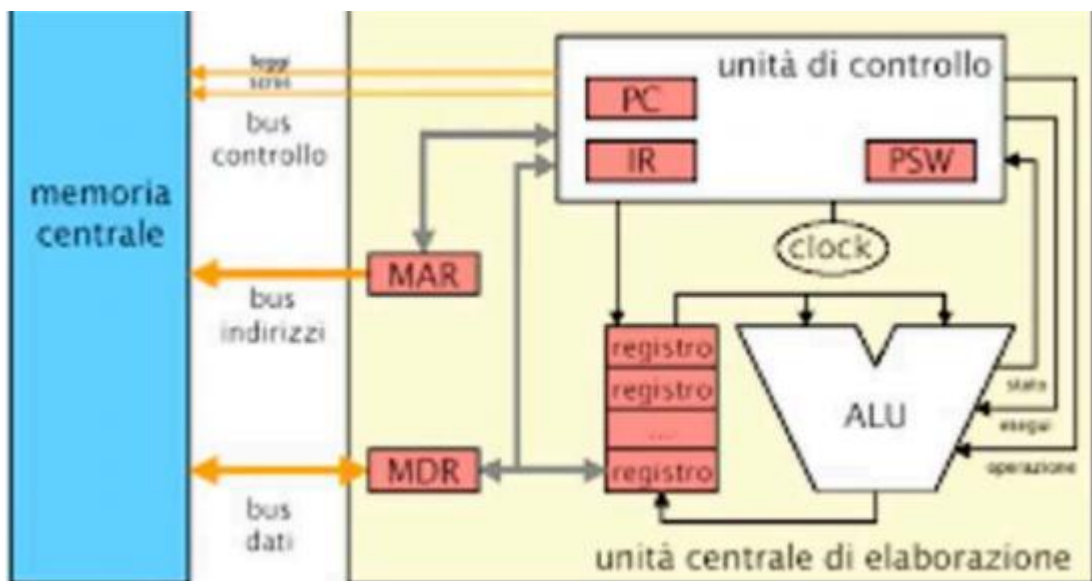
ORALE

1. Architettura di un elaboratore #@

Collegati al bus: CPU, RAM, Disco, Tastiera, Video, interfaccia di un dispositivo esterno

Il bus contiene circa un centinaio di fili, di input/output tra diversi dispositivi, questi si dividono in fili di indirizzo, dati e controllo. La comunicazione è di tipo broadcast ed è compito del ricevente della transazione riconoscersi come il destinatario, solo un dispositivo può essere mittente di un messaggio(master) e possono esserci n riceventi (slave).

Focus su collegamento memoria/cpu



MAR = memory address register: contiene indirizzo di accesso in memoria

MDR = memory data register: contiene dati letti/da scrivere in memoria

PC/IP = program counter/instruction pointer: contiene indirizzo della prossima istruzione

IR = instruction register: contiene ultima istruzione prelevata

PSW= program status word: contiene i flag e livello di privilegio

La Cpu funziona con un ciclo di funzionamento

Fetch → decode → esecuzione → Fetch → ... a fine di ogni fase di interruzione la CPU controlla se ci sono interruzioni da gestire

Il microprocessore mEDU utilizza:

Due spazi di indirizzamento separati per memoria e I/O con indirizzi leggibili da $[0; 2^n - 1]$ con $n_{mem} > n_{I/O}$, nello spazio di I/O sono mappati 3 registri di interfaccia dei dispositivi: controllo (cpu→disp), stato (disp→cpu), buffer (cpu↔disp).

Due registri di stato: IP e F.

Quattro registri generali da 2 byte: AX, BX, CX, DX. Ognuno dei quali è accessibile nella sua parte superiore e inferiore (AH-AL, BH-BL,...).

Un registro SP di puntatore allo stack.

Le **istruzioni** sono di tipo RISC, ovvero istruzioni semplici tutte lunghe 1 byte, il che ne permette l'esecuzione in pipeline.

Memoria cache

Memoria più piccola ma veloce che si posiziona tra cache e bus con il compito di velocizzare le operazioni di accesso in memoria. È divisa in blocchi (chiamate cacheline), quando la cpu vuole accedere ad un indirizzo la cpu controlla se ha quell'indirizzo (o meglio il blocco contenente quell'indirizzo), in caso affermativo si occupa direttamente di rispondere alla transazione, altrimenti propaga l'operazione in memoria e poi copia il blocco. La velocità media della cache si può calcolare come $V = \text{HitRate} * V_{\text{hit}} + \text{MissRate} * V_{\text{miss}}$.

Un singolo elemento della cache è del tipo V | TAG | Contenuto dove V è un bit di validità che indica se ciò che è scritto in tag e contenuto ha significato o meno, TAG indica l'identificatore del blocco e poi il contenuto può essere istruzione o dato (esistono anche cache specializzate per solo istruzioni o solo dati).

In ordine crescente di dimensioni e decrescente di velocità si possono categorizzare le memorie secondo una **gerarchia** specifica:

registri->cache->RAM->disco->nastro. Le prime 3 sono volatili, le ultime due persistenti sono usate per lo swap dei processi.

I/O, interruzioni e DMA

Al bus non viene collegato il dispositivo direttamente ma un'interfaccia (o controllore) che si occupa di adottare il protocollo della macchina, è accessibile attraverso 3 registri: controllo, stato e buffer. Dato che il tempo di lavoro dei dispositivi di I/O (IO burst) è >> del tempo di lavoro della CPU si può permettere al dispositivo di lavorare in parallelo alla CPU che nel frattempo svolgerà altre operazioni. Oppure si può mettere la cpu in busy-wait dove controlla internamente ad un ciclo se l'operazione è conclusa o no, ma questo risulta una perdita di tempo.

Quando il dispositivo esterno ha completato il suo lavoro invia un'interruzione alla CPU che alla fine della sua ultima di esecuzione la nota, salterà quindi all'indirizzo associato a quel determinato tipo di interruzione nella Interrupt Descriptio Table dove si trova la routine associata e un vettore di interruzione contenente l'indirizzo della routine e il registro psw con il bit di privilegio settato al livello supervisore. È importante notare che le routine non possono terminare con RET perché è necessario recuperare in pila oltre il precedente valore di IP, anche il precedente registro PSW, per questo viene utilizzata un'istruzione IRETQ.

Esistono 3 tipi di interruzione: interruzioni interne (o software) sono chiamate direttamente dal programmatore e vengono usate principalmente per operazioni di I/O o di accesso a strutture dati condivise, interruzioni esterne sono quelle dei dispositivi e le eccezioni sono causate a tempo di esecuzione in caso di comportamento imprevisto.

È possibile permettere al dispositivo di dialogare direttamente con la RAM attraverso il meccanismo di DMA (direct memory access) nei momenti in cui il bus non è occupato dalla CPU, per ciò sono necessari due registri nel dispositivo: un contatore del numero di operazioni da svolgere e un puntatore all'indirizzo a cui accedere

2. I 3 metodi per risolvere la mutua esclusione, pro e contro (prologo-epilogo, lock-unlock, semafori) #@

- a. Il problema di mutua esclusione si presenta quando più processi cercano di accedere in maniera concorrente alla stessa risorsa condivisa, la quale ovviamente non può essere utilizzata in contemporanea perché potrebbe portarsi ad uno stato inconsistente. Per questo gli accessi a queste risorse devono essere atomici. Sono state ideate 3 soluzioni:

Prologo/epilogo

Prologo:

```
while(occupato)
    occupato = 1;    //lock
<sezione critica>
```

Epilogo

```
Occupato = 0;    //unlock
```

Problema:

la variabile "occupato" è essa stessa una risorsa condivisa, e dovrebbe avere un proprio meccanismo di mutua esclusione

Istruzione TEST-and-SET

```
lock:
    TSL registro,x    //copia x in registro e pone x = 1
    CMP registro,0
    JNE lock
    RET
unlock:
    MOV x, 0
    RET
```

PRO:

TSL è istruzione in assembly, atomica perché interruzioni non vengono controllate in mezzo ad una fase di esecuzione.

CONTRO: busy-wait

```
//nel codice in c
lock(x);
    <sezione critica>
unlock(x);
```

Semafori MUTEX@

Un semaforo è una classe a cui sono associati un valore e una coda di attesa di processi, su cui sono definite due operazioni: signal, wait

```
void wait(s){
    if(!s.value){    //se l'intero vale 0
        <processo si sospende e si blocca in s.queue>
        <chiama scheduler>
    }
    else
        s.value--;
}
```

```
void signal(s){
    if(s.queue) //se ci sono processi in coda
        <estrai il processo in testa ed inseriscilo in coda
pronti> //estrazione FIFO previene la starvation
    else
        s.value++;
}
```

La particolarità del semaforo MUTEX è che viene inizializzato con s.value = 1, il che permette l'accesso alla risorsa solo ad un processo per volta

```
wait(mutex);
    <sezione critica>
signal(mutex);
```

3. Device driver del timer #@

Il **decruttore** è composto dai seguenti campi:

indirizzo dei 3 registri: controllo, stato e contatore

fine_attesa[N] → N semafori utilizzati per bloccare l'iesimo processo (ci sono N processi

ritardo[N] → N interi contenente il numero di quanti di tempo rimanente dall' i-esimo processo bloccato

fornisce anche una **primitiva** che fornisce al processo di sospendersi per del tempo

```
void deelay(int ritardo){
    int proc;
    proc = <indice del processo in esecuzione>;
    descrittore.ritardo[proc] = ritardo;
    descr.fine_attesa[proc].wait()
}
```

La procedura di **interruzione** invece si occupa di decrementare il tempo di attesa dei processi e vegliarli se arriva a 0

```
void inth(){
    for(int i = 0; i<N; i++){
        if(descr.ritardo[i] != 0){
            descr.ritardo[i]--;
            if(descr.ritardo[i] == 0)
                descr.fine_attesa[i].signal;
        }
    }
}
```

4. Modalità di accesso ad un file nel disco #@

Per accedere ad un file nel file-system è necessaria una chiave: l' "I-Number", che altro non è che l'indice dell' "I-Node" nella "I-List", è necessario localizzare il file nel disco e spostarlo in memoria. Nella fase di apertura oltre che spostare l' I-node in memoria principale spostato anche parte del file ("memory mapping"). Nella fase di chiusura risposto il file e l'I-Node in memoria di massa.

Immaginiamo ogni file come una sequenza di unità di lettura/scrittura chiamate record.

Ci sono 3 modalità di accesso ad un file nel disco

Sequenziale:

inizia sempre dal primo record del file e lo scorre fino al record interessato. Le operazioni di lettura e scrittura definite sono: `readnext(f, &v)` e `writenext(f, v)`. La prima salva nella variabile `v` il record successivo da quello puntato attualmente dall'I/O pointer del file `f` e incrementa il puntatore di un record, il secondo scrive nel successivo record del file `f` rispetto all'I/O pointer il record contenuto in `v`, successivamente incrementa l'I/O pointer.

Diretto:

posso accedere direttamente a qualunque record `i` del file. Le primitive sono: `read(f, i, &v)` e `write(f, i, v)`. Leggono/scrivono l'`i`-esimo record del file `f` e lo copia in/da `v`. particolarmente conveniente per file di grandi dimensioni, risulta inoltre inutile un puntatore al file aperto in quanto non è necessario alcun riferimento ad un eventuale record corrente

Indice:

questo metodo prevede che un record sia distinto in due campi: chiave e contenuto, a ogni file viene associata una tabella con le corrispondenze chiave-indice del record, che permette l'accesso diretto al record nel file. Le primitive sono `readk(f, key, &v)` e `writek(f, key, v)`, con funzionamento analogo all'accesso diretto.

5. Modalità di allocazione di file su disco #@

Lo spazio disponibile nel disco è composto da una sequenza di blocchi, di dimensione fissa, contenenti molti record, il numero di record per blocco (N_b) = Dimensioni blocco / dimensioni record

Esistono 3 metodi di allocazione di file nel disco

Contigua

Prevede che ogni file sia salvato su una serie di blocchi contigui, questo permette un'efficiente implementazione dei metodi di accesso sequenziale e diretto. Se il primo risulta scontato il secondo invece è reale in quanto basta mantenere all'interno del descrittore del file l'indirizzo del primo blocco del file sul disco (`B`). e l'`i`-esimo record si troverà all'indirizzo $B + i/N_b$.

Di contro però questo metodo ha due problemi. Il primo è che per allocare un file vi è bisogno di una sequenza di blocchi contigui lunga almeno quanto il numero di blocchi del file, è poi possibile utilizzare un algoritmo `best-fit`, `first-fit`, `worst-fit`. Il secondo è una conseguenza del primo, ovvero la frammentazione esterna, che si forma quando il numero di blocchi contigui liberi è molto basso. Per arginare questo problema si può usare un algoritmo di compattamento per spostare in zone contigue tutti i file e creare sequenze di blocchi liberi più lunghe, questo comporta però un importante overhead.

Lista concatenata

I blocchi relativi ad un determinato file possono essere salvati in blocchi qualsiasi, per accedere al primo blocco si inserisce un puntatore nel descrittore del file, e ogni blocco contiene un puntatore al blocco successivo. Questo implica sempre un'implementazione efficiente del metodo ad accesso sequenziale, ma invece un accesso diretto oneroso dato che prima di arrivare al blocco corretto saranno effettuati i/Nb accessi in lettura ai puntatori. Come contro però questo metodo provoca una debolezza strutturale del file system, in caso accadesse la corruzione di un blocco si perderebbe qualsiasi informazione riguardante tutti i blocchi successivi. Questo problema è arginabile utilizzando una lista doppiamente concatenata nei due versi, ed inserendo nel descrittore di file un puntatore all' primo e uno all'ultimo blocco. Questo comporta comunque maggiore occupazione di memoria e costo di allocazione. Alcuni sistemi (MS/DOS) affiancano alla lista concatenata una tabella chiamata FAT (file allocation table) contenente un elemento per ogni blocco per recuperare il blocco perso. Inoltre spostando la tabella in memoria principale o in cache può anche essere velocizzato di molto l'accesso diretto.

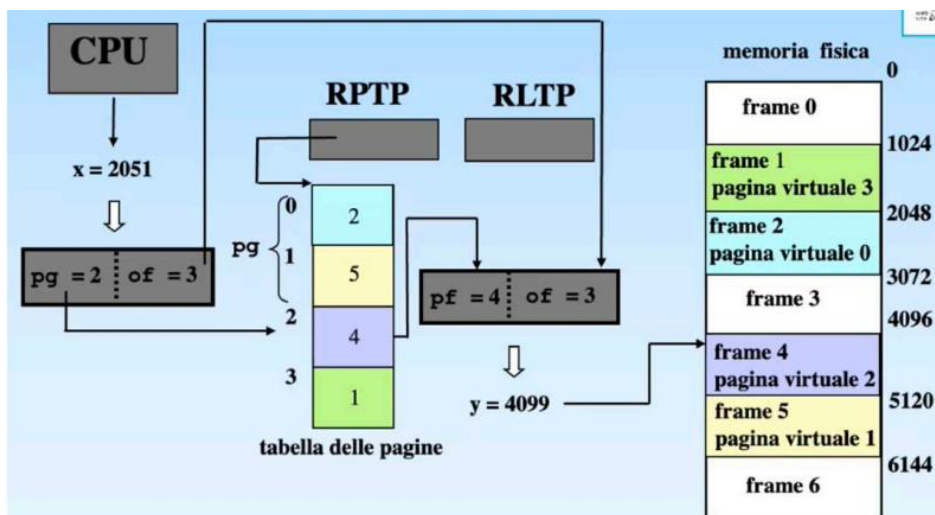
Indice

ad ogni file viene associato un blocco contenente tutti i puntatori ai blocchi in cui sono all'ocati gli effettivi record del file. L'indirizzo del blocco indice è salvato nel descrittore di file. Non necessitando di blocchi contigui non esiste frammentazione. L'unico contro è che il numero di puntatori contenuti nel blocco indice è limitato, e per cui risultano limitate le dimensioni del file, risolvibile utilizzando più livelli di puntatori.

6. Paginazione # @

Nella paginazione gli spazi virtuali e fisici vengono divisi in pagine e frame di medesima dimensione ogni pagina virtuale viene poi associata ad un frame fisico, eliminando la frammentazione esterna dato che dal punto di vista del processo la sua memoria risulta comunque come un unico spazio contiguo. Rimane comunque la frammentazione interna, con spazio sprecato medio $= \frac{1}{2} * \text{Dim frame} * \# \text{processi}$.

Indirizzo virtuale generato è suddiviso in bit di pagina, che indica il numero di pagina e bit di offset, questi sono divisi via hardware, il che comporta che uno stesso programma può girare su dispositivi con pagine di dimensione diversa



La scelta della dimensione della pagine risulta un trade-off tra pro e contro di grandi e piccole dimensioni:

con pagine di piccole dimensione la non contiguità delle pagine aumenta, fino alla sua massima espressione con pagine di 1 byte, al contempo però aumenta in maniera direttamente proporzionale la dimensione della tabella delle pagine, fino a farla diventare al limite grande quanto la memoria virtuale stessa.

Pagine troppo grandi invece aumentano la frammentazione interna, dato che l'ultima pagina di un processo viene usata mediamente a metà, per cui maggiore è la dimensione della pagina maggiore è lo spazio sprecato.

Alla fine si opta per pagine che variano di dimensione tra 512 byte e 4 kbyte

Si può fare controllo sugli indirizzi virtuali generati controllando che il numero di pagina generato sia $\leq$ al contenuto del registro PTLR, contenente il numero di pagine del processo in esecuzione, possono inoltre essere inseriti dei bit di controllo per i diritti di accesso, ad esempio in lettura e scrittura.

È possibile anche condividere delle pagine tra diversi processi con le stesse corrispondenze tra pagina virtuale e fisica

Dato che la traduzione delle pagine in frame è lenta viene mantenuta della MMU una memoria delle ultime traduzioni in tutto e per tutto come se fosse una cache: il TLB (translation lookaside buffer)

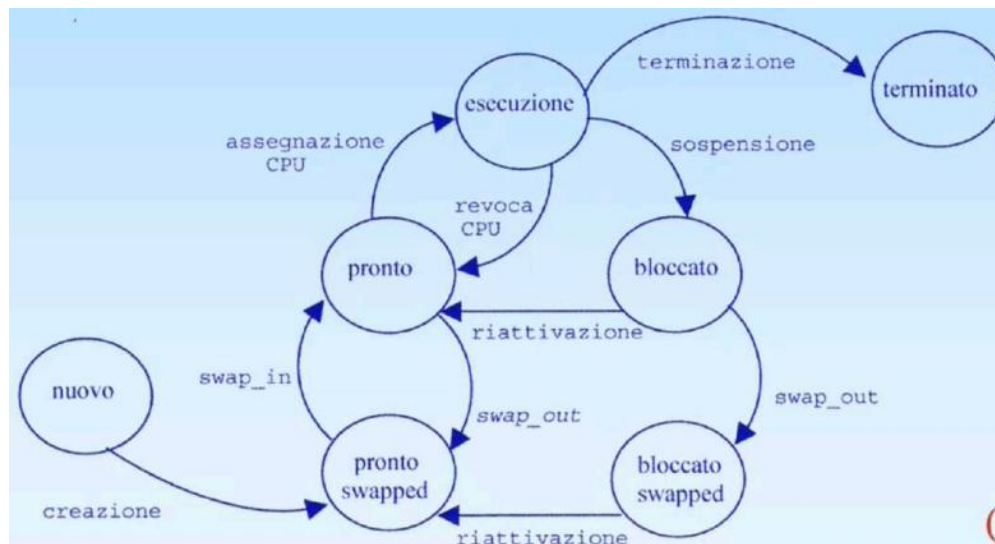
7. Coda multilevel feedback-queue # @

La coda dei processi pronti può essere gestita in molte maniere: con algoritmi pre-emptive o non pre-emptive, prioritari o non prioritari. È possibile anche implementare la coda come un sistema di più code, gestite anche con algoritmi diversi. Ad esempio potrei avere la coda di livello 0 gestita con RR e la coda di livello 1 gestita con FCFS. Questo però potrebbe creare starvation sulla coda 1 oppure creare ritardo sulla coda 0. Rendo quindi il sistema complessivo pre-emptive, facendo pre-emption ogni volta che un nuovo processo entra in coda. Questa implementazione è utile ad esempio per la suddivisione di processi di foreground e di background. Un nuovo processo entra nella coda 0 come processo di foreground, se entro un ciclo non riesce a completarsi/ sospendersi entra nella seconda coda e diventa di background.

La coda **multilevel feedback queue** viene implementata con n code RR e una FCFS con il tempo di RR crescente man mano che aumenta il livello della coda. Un processo pronto entra nella coda 0 con Q0 più basso di tutti, se non viene completato entro un ciclo passa alla coda successiva e così via. In questo modo il sistema auto-regola la priorità delle code e dei processi. Rimane il problema della starvation ma è risolvibile tramite aging: un processo rimasto in coda per troppo tempo rientra nella coda 0 al raggiungimento di un time-out

8. Stati di un processo e stati swapped # @

Gli stati di un processo in un sistema multiprogrammato e il loro spostamento seguono il seguente schema



Quando un processo viene attivato entra in coda pronti, un volta che il processo attualmente in esecuzione viene: terminato, sospeso o viene fatta pre-emption (non presente in tutti i sistemi) allora viene schedulato (estratto dalla testa e inserito in esecuzione) un processo dalla coda pronti. Un processo viene riattivato tramite alcuni eventi che possono essere l'arrivo di un'interruzione esterna oppure viene liberato da una coda di un semaforo da un altro processo.

Lo stato zombie esiste giusto in caso un processo padre desideri informazioni sulla terminazione dei figli.

Gli stati swapped vengono inseriti a seguito della creazione di segmentazione e definiscono se un processo si trova in memoria principale o no. Questi stati si perdono con la segmentazione a domanda dato che non ha senso definire un processo swappato o no se si trova parte in memoria principale e parte in secondaria.

I sistemi unix però hanno un singolo stato swapped per processi interamente non in RAM

9. Scheduling a breve medio e lungo termine#@

Lo scheduling a breve termine è l'algoritmo che decide quale processo prelevare dalla coda pronti ed inserire in esecuzione. Viene invocato spesso, nell'ordine dei ms.

Lo scheduling a medio termine implementa la virtualizzazione decidendo quale e quanto codice passare da memoria principale alla memoria secondaria (swap-in e swap-out di processi parzialmente eseguiti)

Lo scheduling a lungo termine gestisce il grado di multiprogrammazione e decide in un sistema di tipo batch quali tra tutti i programmi in memoria di massa inserire in coda pronti. Interviene di rado, nell'ordine dei secondi o addirittura minuti

10. Cosa si intende per stato sicuro #@

Uno stato è definito sicuro se non può causare deadlock, ovvero se anche solo una delle condizioni necessarie al deadlock è falsa. Le condizioni sono:

- 1) Risorsa a cui si vuole accedere richiede la mutua esclusione

- 2) Un processo in attesa di una risorsa non disponibile si blocca senza rilasciare quelle già acquisite (possesso e attesa)
- 3) Mancanza del diritto di revoca → il s.o. non può revocare risorse ad un processo
- 4) Ciclo nel grafo possesso-attesa → questa condizione diventa sufficiente se è presente una sola istanza per ogni risorsa

11. Come è fatto un device driver # @

Il device driver è il modulo del sistema dedicato alla gestione del dispositivo. Contiene alcune informazioni riguardanti il dispositivo e le sue funzioni di accesso. Consente la comunicazione tra processo applicativo e il dispositivo.

Le informazioni riguardanti il dispositivo si trovano nel **descrittore del dispositivo**, che è generalizzabile in una classe contenente i seguenti campi:

3 campi contenenti l'indirizzo dei registri di stato, controllo e dati del controllore

Semaforo di sincronizzazione tra processo e dispositivo

Contatore del numero di dati da trasferire

Puntatore al buffer per il trasferimento dei dati

Esito del trasferimento

Una funzione di accesso può essere la **read**, si presenta così:

```
int read(int disp, char* pbuf, int cont){
    descrittore[disp].contatore = cont;
    descrittore[disp].puntatore = pbuf;
    <attivazione del dispositivo> --> bit di start nel registro di controllo= 1

    //il processo si sospende nel semaforo di sincronizzazione
    descrittore[disp].dato_disponibile.wait();

    if(descrittore[disp].esito == <codice di errore>)
        return(-1);
    return(cont - descrittore[disp].contatore)
}
```

È compito della read inizializzare il contatore e il puntatore al buffer e far partire il processo esterno. Dopodiché sospende il processo in attesa del risultato e ritorna il numero di byte letti

Il comportamento del dispositivo è descrivibile come l'esecuzione di un processo (processo esterno) ciclico:

```
while(true){
    do{}while(start == 0);
    <esegue comando>
    <registra esito, setta flag = 1>
}
```

Infine all'arrivo dell'interruzione parte la funzione `inth:@`

```
void inth(){
    char b;
    <legge registro di stato>
    if(<bit di errore == 0>){    //se non ci sono errori
        b = <lettura dati>;
        *descrittore[disp].puntatore = b;
        descrittore[disp].puntatore++;
        descrittore[disp].contatore--;
        if(descrittore[disp].contatore != 0)    //se non ho finito
            <riattiva dispositivo start = 1>
        else{
            descrittore[disp].esito = <codice terminazione corretta>;
            <disattiva dispositivo>
            descrittore[disp].dato_disponibile.signal();    //risveglio il processo
        }
    }
    else{
        <routine di gestione errore>
        if(<errore irrecoverabile>)
            descrittore[disp].esito = <codice errore>;
        descrittore[disp].dato_disponibile.signal();    //risveglio il processo
    }
    return;
}
```

La routine di interruzione si occupa di registrare il byte letto e l'esito, se necessario riavviare il dispositivo, e gestire eventuale errore. In ogni caso al termine dell'esecuzione del dispositivo, che sia per un errore e perché ha finito le sue operazioni sveglia il processo applicativo

12. Problema produttore-consumatore# @

Il problema di produttore consumatore è un problema di sincronizzazione e mutua esclusione tra processi verso un buffer condiviso in cui un processo scrive e l'altro legge, con la restrizione che il produttore non può produrre se l'altro non ha ancora letto l'ultimo messaggio e viceversa.

Nel caso semplice con un solo produttore, un solo lettore e buffer contenente un singolo messaggio bastano due semafori: `spazio_disp(S0 = 1)` e `msg_disp(S0 = 0)`

```
produttore:
while(true){
    <produci msg_in>
    wait(spazio_disp);
    buff = msg_in;
    signal(msg_disp);
}

consumatore:
while(true){
    wait(msg_disp);
    msg_out = buff
    <elabora msg_out>
    signal(spazio_disp);
}
```

Sposto spostare elaborazione di `msg_out` dopo la signal per ottimizzare per il produttore

Se invece ammetto buff[n] circolare, k produttori e h consumatori è necessario un ulteriore semaforo di mutex per evitare l'accesso a più produttori e lettori insieme

```

produttore:                                consumatore:
while(true){                                while(true){
    <produci msg_in>                          wait(msg_disp);
    wait(spazio_disp);                        wait(mutex);
    wait(mutex);                             msg_out=buf[testa];
    buf[coda] = msg_in;                      testa = (testa + 1) % n;
    coda = (coda + 1) % n                    signal(mutex);
    signal(mutex);                           <elabora msg_out>
    signal(msg_disp);                        signal(spazio_disp);
}                                            }

```

Posso utilizzare due mutex diversi per rilassare l'ipotesi di mutua esclusione tra produttori e consumatori

13. Deadlock avoidance e Algoritmo del banchiere(singola e multipla istanza) #@

La deadlock avoidance è un tipo di prevenzione dinamica che controlla ad ogni richiesta di acquisizione di una risorsa da parte di processo se lo stato del sistema a seguito dell'assegnazione di tale risorsa è safe o meno

L'algoritmo del banchiere viene utilizzato per risorse a multipla istanza:

```

banchiere(n, m, available[], max[], allocation[], need[]){
/*
    n = #processi;
    m = # tipi di risorse;
    available[m] --> available[i] = k      //disponibili k istanze per la risorsa i
    max[n*m] --> max[i][j] = kc           //Pi necessita al massimo di k istanze di j
    allocation[n*m] --> allocation[i][j] = k //Pi ha k istanze della risorsa j
    need[n*m] --> need[i][j] = k         //Pi necessita di k istanze della risorsa j
                                   = max[i][j] - allocation[i][j]
*/
work[m] = available[m];
finish[n] = false; //per ogni elemento di finish[n]

do{
    <cerca i che garantisca finish[i] = false && need[i][j] <= work[j] per ogni j>
    if(<trovato i>){ //il processo può terminare → liberare le sue risorse
        work[j] = work[j] + allocation[i][j] per ogni j
        finish[i] = true
    }
}while(<trovato i>)

if(finish[i] = true per ogni i)
    return safe;
return not_safe
}

```

Dato che l'algoritmo fa solo previsioni non è necessario modificare realmente il valore di allocation all'interno dell'algoritmo.

Se richiesta > need → errore, se richiesta > available → sospensione

L'algoritmo necessita di sapere $\max[i][j]$, plausibile nei sistemi real-time ma non nei general purpose

L'algoritmo del banchiere nasce per risorse a multipla istanza, ma si può semplificare per risorse a singola istanza. Basta che le matrici e vettori diventino binari.

Per risorse a singola istanza rimane comunque più efficiente il metodo della ricerca di cicli nel grafo "wait-for" o assegnazione di risorse. Se assegnando la risorsa ad un processo si crea deadlock allora si avrà un deadlock

14. Perché si usa l'algoritmo di second chance per approssimare LRU#@

Parlando di algoritmi di rimpiazzamento delle pagine di un processo, secondo i principi di località spaziale e temporale il miglior algoritmo teorico è un algoritmo che sostituisce esclusivamente pagine non più necessarie o che comunque verranno riferite più tardi tra tutte nel futuro, ma siccome le macchine veggenti non sono ancora state inventate ci si accontenterebbe del LRU (last recently used), che però è pesante in quanto necessita di aggiungere 32 bit ad ogni descrittore di pagine e accedere ad ogni generazione di indirizzo, il che introduce molto overhead. Risulta quindi più facile approssimare l'algoritmo tramite un fcfs, oppure più similmente l'algoritmo di second chance (o dell'orologio)

Le pagine vengono controllate in ordine di caricamento, se sono state usate il bit $u = 1$, se $u = 0$ è spostata la variabile "vittima" alla pagina successiva, ovviamente una volta controllata l'ultima pagina riparte dalla prima. La prima pagina che trovo con $u = 0$ è quella che rimpiazzerò. Posso anche inserire il controllo del bit m per evitare pagine modificate ($m=1$), così facendo posso fare swap-in della nuova pagina senza fare swap-out della pagina sostituita, diminuendo leggermente l'overhead. Alla peggio l'algoritmo fa un ciclo completo.

Sarebbe possibile anche scegliere tra le pagine da rimpiazzare pagine di altri processi non in esecuzione (rimpiazzamento globale), questo offrirebbe maggiore flessibilità, ma provocherebbe un maggior numero di page-fault nei processi a cui sono state rimpiazzate le pagine.

Questo algoritmo evita il **trashing**, ovvero l'attività in cui la CPU è sostanzialmente impegnata esclusivamente allo swap di pagine

15. A cosa serve la tabella delle pagine #@

La tabella delle pagine serve a controllare la corrispondenza pagina-frame, non contiene infatti la corrispondenza tra singoli indirizzi, dato che ciò richiederebbe una tabella grande quanto l'intera memoria virtuale.

L'indirizzo fisico si ottiene con i bit meno significativi (offset) = a quello dell'indirizzo virtuale, e i più significativi sono il numero di frame letto nella tabella.

Un singolo elemento della tabella contiene oltre all'indice della pagina fisica anche due bit R, W per il controllo dell'accesso in lettura e/o scrittura

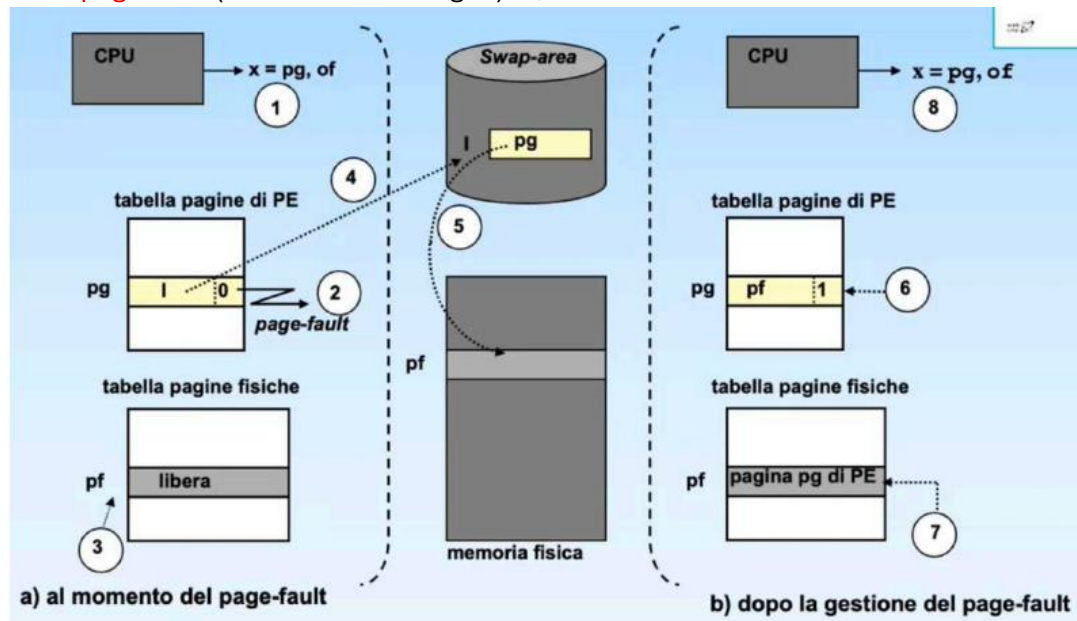
Per permettere la paginazione a domanda nel campo di controllo sono inseriti anche i bit P, U, M per indicare se la pagina si trova in memoria principale, se sono stati generati suoi indirizzi o se la pagina è stata modificata. Se $P=0$ nel campo della pagina fisica è presente l'indirizzo su disco della swap-area

16. Cosa c'è scritto nel descrittore di frame #@

Nel descrittore di frame sono presenti due campi:

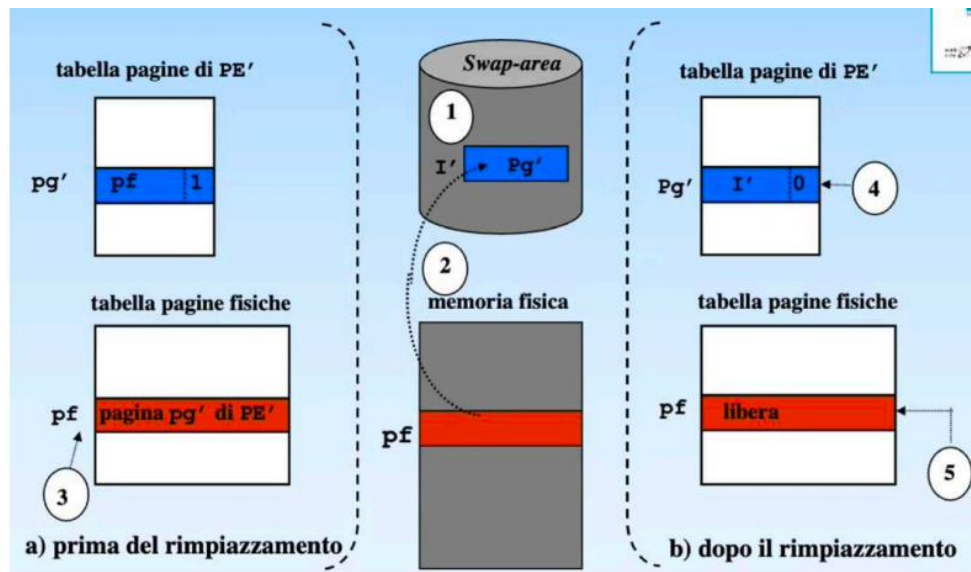
il primo indica se la pagina è libera o, se occupata, l'indice del processo e della pagina virtuale che lo occupano. Il secondo è un bit di lock, che impedisce il rimpiazzamento di quella pagina

17. Gestione del page-fault (descrizione + disegno) #@



- 1) Generazione dell'indirizzo virtuale di pagina pg
- 2) Il bit di presenza nella tabella delle pagine $P=0 \rightarrow$ page-fault
- 3) Dalla tabella delle pagine fisiche viene ricavato l'indice pf di un frame libero pf
 - a. Se non sono stati trovati frame liberi esegue *rimpiazzamento*
- 4) Dalla tabella delle pagine ricavo l'indirizzo della swap area della pagina pg richiesta
- 5) Avvio il trasferimento della pagina dalla memoria di massa alla memoria principale
----sospendo il processo che si riattiverà dopo la conclusione del trasferimento----
- 6) Aggiorno i campi di indice pg della tabella delle pagine
- 7) Aggiorno il campo pf della tabella dei frame con la pagina pg
- 8) Riesegue l'istruzione che ha generato page-fault

Algoritmo di rimpiazzamento:



- 1) Ricerca nella swap-area un'area disponibile all'indirizzo I' nella quale trasferire il contenuto del frame pf da rimpiazzare
- 2) Trasferimento da memoria fisica a memoria di massa
 ----sospendo il processo che si riattiverà dopo la conclusione del trasferimento----
- 3) Dalla tabella dei frame ricavo l'indice del processo PE' a cui apparteneva la pagina pg' rimpiazzata
- 4) Pongo a 0 il bit P dell'entrata relativa alla pagina pg' della tabella delle pagine di PE' e scrivo l'indirizzo I'
- 5) Nella tabella dei frame scrivo che la pagina pf è diventata libera

n.b: può essere importante non scegliere alcune pagine per il rimpiazzamento, ad esempio quelle in cui un dispositivo stava facendo DMA con un altro processo, il dispositivo scriverebbe sempre agli stessi indirizzi fisici andando in collisione con la pagina del nuovo processo. Per questo nel descrittore di frame c'è un bit di lock per evitare che quella pagina venga scelta come rimpiazzamento

18. Cosa è una condition variable #@

Una condition variable è una variabile sul quale un processo può bloccarsi in attesa dell'avvenimento di un determinato evento tramite due operazioni. Sono definite due operazioni per questo meccanismo: wait e signal. La prima sospende il processo fino all'arrivo di una signal, la seconda sveglia uno (o più processi) dalla coda della condition variable e permette al processo svegliato di andare in esecuzione (signal and wait) o continua il processo attualmente in esecuzione (signal and continue)

19. Perché si controlla che il numero di segmento generato dalla CPU sia minore del numero di segmenti presenti nella tabella dei segmenti? E che l'offset sia minore della dimensione? #@

Perché in caso contrario viene sollevata un'eccezione di protezione a seguito del tentativo di accesso ad un segmento inesistente.

Analogamente se offset generato è maggiore della dimensione del segmento vuol dire un tentativo di accesso ad un'informazione non appartenente al segmento, e viene

sollevata una nuova eccezione, che però non rappresenta automaticamente un errore, può essere anche interpretato come una richiesta di necessità di espansione del segmento

20. Quali sono i campi della tabella dei segmenti? #

Sono presenti i campi base e limite che indicano indirizzo di base partizione(segmento) e la sua dimensione, sono inoltre presenti i bit di controllo R,W per l'accesso in lettura/scrittura sul segmento e altri 3 bit utili per il caricamento a domanda): P per indicare se la partizione si trova in memoria principale o no, U per vedere se è stato generato un indirizzo da quella partizione e M per sapere se è stata modificata.

21. Registri collegati alla tabella dei segmenti e delle pagine#@

STLR (segment table limit register) = contiene la dimensione della tabella dei segmenti (ovvero il numero dei segmenti)

STBR(segment table base register) = contiene l'indirizzo di base della tabella dei segmenti

RPTP(registro puntatore della tabella delle pagine) = contiene l'indirizzo di base della tabella delle pagine

RLTP(registro lunghezza tabella pagine) = contiene la dimensione della tabella delle pagine

Inoltre sono presenti dei registri nel TLB che contengono le traduzioni dirette tra indirizzi virtuali e fisici

22. Dove sono implementati fisicamente i registri della periferica #@

Nel controllore della periferica, i principali sono 3:

controllo: usato dalla cpu per comandare il dispositivo

stato: usato dal dispositivo per comunicare il proprio stato

dati: usato da entrambi

23. Come vanno inizializzati i semafori nel descrittore del timer#@

I semafori del descrittore del timer sono tutti inizializzati a 0, perché utilizzati per bloccare il corrispettivo processo durante l'utilizzo della primitiva deelay, che chiama una wait sul semaforo per bloccare il processo, se fossero inizializzati ad un valore > 0 il processo non si suspenderebbe.

24. Confronto tra algoritmi di scheduling #@

Esistono diversi tipi di algoritmi di scheduling, si dividono in pre-emptive e non preemptive, e poi in prioritari e non prioritari. Per valutare la validità degli algoritmi vengono presi in considerazione diversi fattori:

utilizzo della cpu → $\text{efficienza} = T_{\text{cpu-busy}} / T_{\text{tot}}$

tempo medio di completamento(turn arund time) → tempo da quando un processo entra in coda pronti la prima volta a quando termina. Si distingue dal tempo di esecuzione perché conta anche il tempo in coda pronti

produttività media → # processi completati in un unità di tempo

tempo di risposta → tempo dal primo ingresso in coda pronti alla fine del primo cpu-burst, senza contare la pre-emption

tempo di attesa → tempo i cui un processo rimane in coda pronti

rispetto della deadline → valido solo per sistemi real-time

FCFS(first come first service)

Algoritmo non pre-emptive, non prioritario: Si attiva quando il processo in esecuzione si sospende o termina.

Turn around medio e tempo di attesa medio grande → se un processo “piccolo” entra in coda dopo un processo “grande” deve attendere tanto tempo. Posso migliorarlo facendo entrare tramite scheduling a lungo termine i processi in ordine di cpu-burst crescente

Overhead molto piccolo.

SJF(shortest job first)

Algoritmo non pre-emptive, prioritario con priorità statica: la priorità è inversamente proporzionale alla durata del cpu_burst, a parità di priorità si usa FIFO. Essendo non pre-empting esegue solo quando la cpu va in idle.

Se la coda dei processi pronti non è ordinata la ricerca del processo da estrarre risulta $O(n)$, ma se è ordinata posso estrarre direttamente dalla testa, a complessità non è scomparsa, l'ho solo spostata nell'algoritmo di inserimento in coda pronti.

Per tempo di attesa e turn around medio si comporta meglio del fcfs.

Rischio di starvation → i processi a bassa priorità potrebbero non eseguire mai

SRTF(shortest remaining time first)

Algoritmo pre-emptive a priorità dinamica: tempo rimanente e priorità sono inversamente proporzionali, essendo pre-emptive esegue quando la cpu va in idle o quando entra in coda pronti un nuovo processo a priorità maggiore, in tal caso il processo attualmente in esecuzione torna in coda pronti.

Posso mantenere la coda ordinata per priorità come nel sjf.

Si può risolvere la starvation tramite l'aging: dopo un po' di tempo che un processo non viene eseguito scatta un time-out e ne aumento la priorità, la resetto all'ingresso di un nuovo processo in coda.

RR(round-robin)

pre-emptive non prioritario time-sharing.

L'algoritmo accede alla coda pronti come FCFS(preleva in testa $O(1)$). Ogni processo può eseguire per un tempo Q prima di essere reinserito in coda alla coda pronti. Per sua natura questo algoritmo non può generare starvation.

Tempo di attesa = somma di tutti i tempi in cui è stato in coda pronti

Attesa media = $n * (c + Q)$ (n = # processi e c = tempo di overhead)

Frequenza di esecuzione di un processo = $Q / \text{attesa media} = 1/n$ (c trascurabile)

Per la valutazione degli algoritmi di schedulazione dei sistemi hard-real time considero la deadline come il tempo del prossimo ingresso in coda pronti del processo, dato che molto spesso sono processi periodici.

PM(period_monotonic)

Algoritmo non pre-emptive prioritario: priorità statica direttamente proporzionale al periodo del processo.

In buona sostanza fa cacare, funziona solo a bassa efficienza.

RM(rate monotonic)

Algoritmo non pre-emptive, prioritario: priorità statica direttamente proporzionale al tempo di rientro in coda pronti.

Schedulabile con efficienza maggiore del pm.

EDF(earliest deadline first)

Algoritmo pre-emptive a priorità dinamica: priorità cambia in base al processo con deadline più vicina, viene ricontrollata ogni volta che un processo entra in coda pronti, a parità di priorità continua il processo attualmente in esecuzione.

25. Modello bell-lapadula e biba#@

Il modello di sicurezza creato da Bell e LaPadula si basa su un sistema multilivello a 4 livelli: non classificato, confidenziale, segreto e top-secret

Si basano su due proprietà fondamentali per mantenere la confidenzialità dei dati:

proprietà di sicurezza semplice in lettura → un soggetto può leggere solo dati di livello uguale o inferiore

proprietà * in scrittura → un soggetto può scrivere solo dati verso un livello superiore

il modello biba ha proprietà in lettura e scrittura opposte, ed è ottimo per il mantenimento dell'integrità dei dati

proprietà di integrità semplice in lettura → un soggetto non può leggere dati a integrità inferiore, evita di leggere dati non affidabili

proprietà * di integrità in scrittura → un soggetto non può scrivere dati verso integrità superiore, evita che soggetti poco affidabili modifichino dati critici

26. Definizione di Rilocalizzazione statica e dinamica #@

La rilocalizzazione statica riloca tutti gli indirizzi da virtuali a fisici prima che il processo inizi la sua esecuzione, durante la fase di caricamento.

Nella rilocalizzazione statica ad ogni indirizzo virtuale v corrisponde sempre e solo lo stesso indirizzo fisico f → $f = v + \text{cost}$.

Una volta che un programma è stato caricato in memoria non può più essere relocato in una posizione diversa in memoria perché il valore scritto nello stack a seguito di un'istruzione call contiene l'indirizzo fisico dell'ultima istruzione dopo la call, se il processo poi viene swappato fuori e di nuovo swap-in una nuova locazione la rete riprendere il vecchio valore fisico dallo stack, ad un indirizzo a cui l'accesso ora non è più disponibile.

La rilocalizzazione dinamica quindi ritarda la rilocalizzazione degli indirizzi in fase di esecuzione, in questo modo la cpu genera indirizzi virtuali, la cui funzione di mappatura agli indirizzi fisici è implementata nella MMU, traducendo dinamicamente gli indirizzi da virtuali a fisici.

Un semplice meccanismo di rilocalizzazione dinamica utilizza i registri base e limite, contenenti l'indirizzo della prima istruzione in memoria fisica e la dimensione della partizione di memoria. La cpu genera un indirizzo x , se questo è $>$ limite viene sollevata un'eccezione, altrimenti vi viene sommata la base e può essere eseguito l'accesso in memoria

27. Memoria partizionata (partizioni fisse/variabili e multiple) #@

Memoria virtuale è costituita da un unico spazio virtuale contiguo, è necessario quindi cercare in memoria una partizione libera di dimensioni $\geq$ all'intera immagine del processo, con il vincolo che tale partizione rimarrà assegnata al processo fino alla terminazione di esso.

Per risolvere questo problema

Partizioni fisse

Suddivido la memoria in $n+1$ partizioni di indirizzo iniziale e dimensioni fisse. La prima contiene la parte residente (sempre in memoria) del sistema, le altre sono per le immagini dei processi.

È altamente improbabile che i processi rientrino perfettamente in una partizione → frammentazione interna. Dato che la rilocalizzazione è statica per ogni partizione è presente un descrittore con una coda che identifica i processi la cui immagine deve essere swappata in quella partizione

Dato che la dimensione delle partizioni è fissa e decisa in fase di installazione del sistema risulta una totale mancanza di flessibilità dell'sistema

Partizioni variabili

Inizialmente esistono solo due partizioni, quella riservata al sistema, ed un'altra, ogni volta che entra un processo viene allocata una partizione delle dimensioni necessarie e quella libera si restringe. Quando un processo termina libera la sua partizione, adesso il numero di partizioni aumenta, a meno che la partizione del processo non fosse adiacente ad una già libera, in quel caso vengono compattate in un'unica partizione di dimensioni maggiori.

Il fatto che ogni processo occupi una partizione della dimensione necessaria elimina la frammentazione interna, ma introduce la frammentazione esterna, è possibile infatti che si entri in uno stato in cui esistono tante piccole partizioni di dimensioni insufficienti per qualsiasi processo, portando comunque a spreco di memoria.

Per ricordare lo stato delle partizioni libere si riserva una locazione nella memoria di sistema con l'indirizzo della prima partizione libera, in ogni partizione libera poi sono presenti nelle prime due locazioni: la dimensione della partizione e il puntatore alla partizione libera successiva.

L'ordine della lista delle partizioni libere può essere deciso a seconda dell'algoritmo che si decide di usare per l'installazione delle partizioni dei processi, con un best-fit (rimane partizione libera piccola) ordino la lista per dimensione crescente ed estraggo dalla prima disponibile, worst-fit (rimane partizione libera grande) ordino la lista per dimensione decrescente ed estraggo dalla testa, first-fit (rimangono partizioni di dimensioni medie) ordino la lista per indirizzo ed estraggo dalla prima disponibile. Best fit inoltre permette più facilmente la fusione di partizioni libere

Partizioni multiple

Per condividere il codice tra due o più processi posso segmentare lo spazio virtuale creando partizioni multiple per un singolo processo, e permettendo l'allocazione di questi segmenti anche in partizioni anche non contigue tra loro (molto simile alla segmentazione, solamente con rilocalizzazione statica)

28. Monitor#

Un monitor è una classe contenente delle strutture dati accessibili esclusivamente attraverso le procedure definite nel monitor stesso, eseguite tutte in mutua esclusione tra loro: ovvero quando una procedura esegue se un altro processo tenta di eseguire una procedura si mette in coda al monitor. Contiene inoltre le conditions_variable, ovvero variabili su cui sono definite le operazioni di signal and wait per sospendere i processi.

Per garantire la mutua esclusione delle procedure sono utilizzati due semafori e un intero: un semaforo mutex(S0 = 0) e un semaforo next(S0 = 0), next_count = contatore dei processi in coda al monitor.

La compilazione di una procedura <F> risulta come:

```
wait(mutex)
<F>
if(next_count > 0)
    signal(next);
else
    signal(mutex);
//solo l'ultimo processo in coda rilascia il mutex
```

Per ogni variabile condition x esiste un semaforo x_sem(S0 = 0) e un intero x_count = contatore dei processi che hanno eseguito una wait

```
//monitor signal-and-wait
x.wait(){
    x_count++;
    if(next_count > 0)
        signal(next);
    else
        signal(mutex);
    wait(x_sem);
    x_count--;
}

x.signal(){
    if(x_count > 0){
        next_count++;
        signal(x_sem);
        wait(next);
        next_count--;
    }
}
```

È possibile anche implementare una condition_wait inserendo un fattore di priorità per l'inserimento nella coda di x

```
//monitor signal-and-continue
x.wait(){
    x_count++;
    if(next_count > 0)
        signal(next);
    else
        signal(mutex);
    wait(x_sem);
}

x.signal(){
    if(x_count > 0)
        signal(x_sem);
}
```

```

x_count--;
next_count++;
sem_wait(next);
next_count--;
}

```

29. Problema dei filosofi (descrizione + implementazione con semafori + perché i semafori non vanno bene + implementazione con monitor) #@

Il problema dei filosofi è un problema di mutua esclusione tra processi, nell'esempio 5 filosofi (processi) seduti ad un tavolo rotondo devono usare ognuno 2 bacchette (risorse condivise) per mangiare, il problema è che la bacchetta alla loro destra è condivisa con il filosofo a destra, quella di sinistra con il filosofo a sinistra.

È possibile implementare una soluzione tramite semafori:

```

chopstick[5]; //array di 5 semafori
do{
    wait(chopstick[i]);    //bacchetta a destra
    wait(chopstick[(i-1)%5]); //bacchetta a sinistra
    <mangia>
    signal(chopstick[(i-1)%5]);
    signal(chopstick[i]);
    <pensa>
}while(true)

```

Questa implementazione però non può portare al deadlock se ogni filosofo (processo) prende subito la bacchetta alla destra, nessuno ha più una bacchetta a sinistra. Per evitare lo stallo o permetto ad un solo processo di usare tutte le risorse necessarie prima di far partire gli altri, ma questa è una soluzione troppo inefficiente, oppure evito wait all'interno delle sezioni critiche. **Utilizzo un monitor**

```

monitor filosfi{
    enum {thinking, hungry, eating} state[5];

    condition self[5];

    void pickup(int i);
    void putdown(int i);
    void test(int i);

    void initialization_code(){ for(int i=0; i<5; i++) state[i] = thinking; };
}

void pickup(int i){
    state[i] = hungry;
    test(i);
    if(state[i] != eating)
        self[i].wait();
}

void putdown(int i){
    state[i] = thinking;
}

```

```

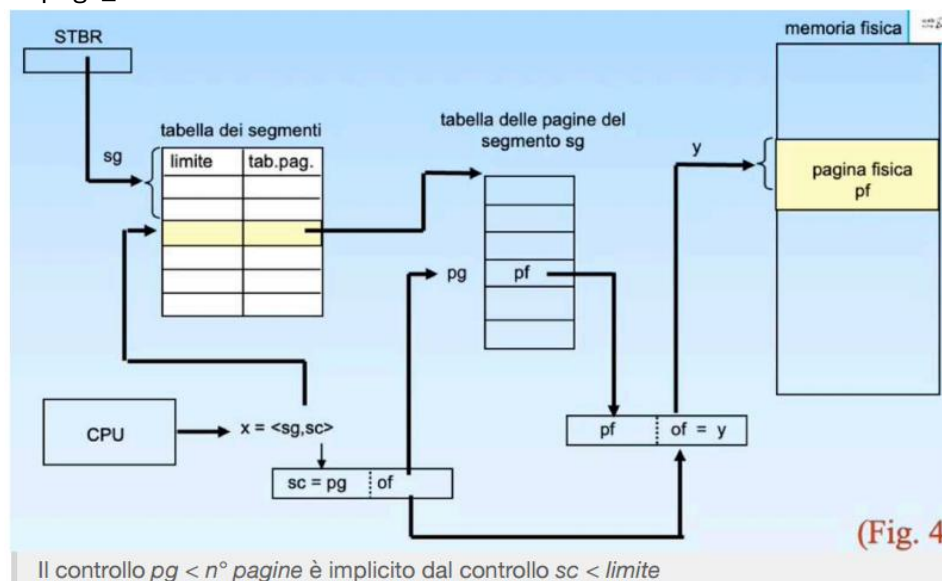
    test((i+4)%5);
    test((i+1) % 5);    //rischio la starvation, dovrei alternare i test
}

void test(int i){
    if(state[(i+4)%5] != eating && state[i] == hungry && state[(i+1)%5] != eating)
        state[i] = eating;
        self[i].signal();
}

```

30. Segmentazione paginata # @

Unisce lo spazio virtuale segmentato della segmentazione alla allocazione non contigua della memoria fisica tipica della paginazione, può causare sia `segment_fault` che `page_fault`



Notare che il campo base della tabella dei segmenti ovviamente non indica più l'indirizzo in memoria della partizione usata, ma l'indirizzo della tabella delle pagine relative a quello specifico segmento.

31. Deadlock prevention #@

La deadlock prevention (o prevenzione statica) compie un'analisi statica del codice per determinare se può o non può generare deadlock. Ci sono diverse azioni possibili per prevenire un deadlock:

negare la mutua esclusione: beh certo, e per dimagrire basta non mangiare.

Negare possesso e attesa: un processo richiede subito tutte le risorse necessarie e si blocca se non le può avere → rischio starvation se il processo ha bassa priorità

Ammettere la pre-emption delle risorse: se un processo si blocca in attesa di una risorsa libera quelle già possedute, il processo si riattiva quando tutte tornano libere. Questo introduce molt overhead: devo mantenere lo stato delle risorse perché il processo potrebbe avere già iniziato a lavorarci

Negare circular-wait: ordino i tipi di risorse secondo un ordine gerarchico e permetto ad un processo di richiedere una risorsa solo se è in possesso di risorse di ordine maggiore o uguale a quella richiesta → vincolo forte sulla struttura del codice.

32. Deadlock detection e correction #@

Esistono degli algoritmi che rivelano la presenza di deadlock che vanno in esecuzione se il fattore di utilizzazione della cpu è inferiore al x% per troppo tempo:

risorse mono istanziate

cerco un ciclo nel grafo wait-for → nodi = processi, arco $P_i \rightarrow P_j$ se P_i attende una risorsa di P_j

risorse pluri-istanziate

simile all'algoritmo del banchiere, ma need viene sostituito da request e finish[i] viene inizializzato già a true se allocation[i][j] = 0 per ogni j, tanto non è rilevante ai fini del controllo del deadlock. Se a fine algoritmo esiste almeno una i t.c finish[i] = false si ha deadlock

metodi di recovery

abortisco tutti i processi in deadlock

abortisco uno dei processi in deadlock → libera le sue risorse, quale scelgo? Chi lascia più risorse? Chi ha meno priorità?...

revoco risorse di alcuni processi → molto overhead, scelgo le vittime in base alla complessità del ripristino dello stato delle risorse

33. Comunicazione nei sistemi a memoria globale e memoria locale(primitive di send e receive) #@

I sistemi a memoria locale comunicano tramite strutture dati condivise a cui hanno accesso in mutua esclusione, i processi di modelli a memoria locale invece hanno ognuno un proprio spazio di indirizzamento separato e la loro comunicazione avviene tramite le primitive di scambio di messaggi send e receive.

Entrambe queste primitive possono essere bloccanti o non bloccanti:

se uso una receive non bloccante è obbligatorio che il processo si blocchi in una busy_wait in attesa dell'arrivo del messaggio, questo però non è un grande problema perché molto probabilmente i processi girano su dispositivi separati o sistemi multiprocessore.

Invece se voglio sapere quando il messaggio è stato ricevuto utilizzando una send asincrona(non bloccante) devo utilizzare subito dopo una receive sincrona(bloccante) in attesa dell'ack.

Il messaggio è formato da intestazione e corpo, nell'intestazione sono presenti: origine, destinazione, tipo di messaggio, lunghezza e info di controllo.

```
produttore:                                consumatore:
do{                                         do{
    produci(&msg);                          receive(mitt, &msg);
    send(dst, msg);                          consuma(msg);
}while(!fine);                             }while(!fine);
```

questo è un esempio di comunicazione diretta, simmetrica: la send arriva direttamente al destinatario che la riceve direttamente dal mittente.

Un modello client-server invece ha una comunicazione di tipo diretto, asimmetrico: ogni client invia il messaggio ad una coda del server, e il server preleva il messaggio dalla coda.

34. Problema dei reader e dei writer # da risolvere ++ lettore@

Immaginando di avere un documento condiviso in cui alcuni processi possono scrivere e altri leggere, sia lettura che scrittura avvengono in mutua esclusione, tra loro e con gli altri.

Sono necessari due semafori e un contatore: $\text{mutex}(S_0 = 1)$, $\text{wrt}(S_0 = 1)$, readcount = contatore di quante persone stanno leggendo;

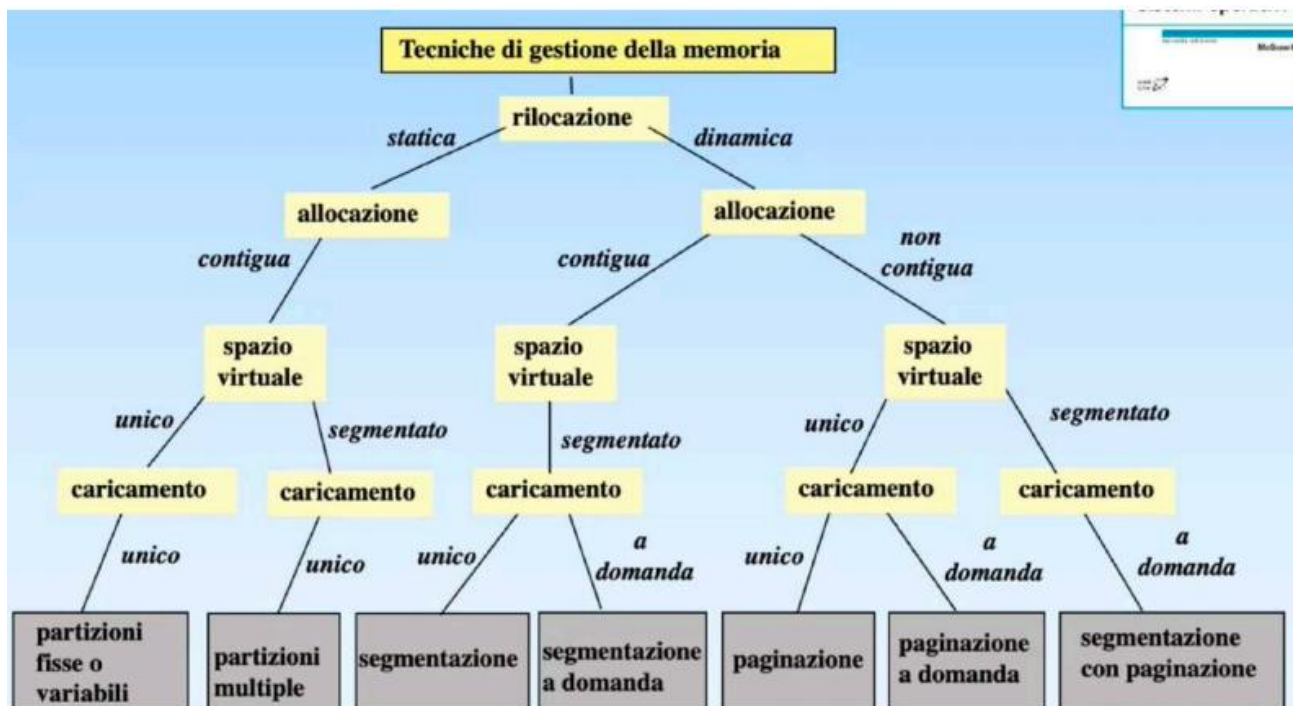
```

lettore:                                scrittore:
do{                                     do{
    wait(mutex);                        wait(wrt);
    readcount++;                        <scrivi>
    if(readcount == 1)                 signal(wrt);
        wait(wrt);                     }while(true);
    signal(mutex);
    <lettura>
    wait(mutex);
    readcount--;
    if(readcount == 0)
        signal(wrt);
    signal(mutex);
}while(!fine);

```

Potrei inserire priorità per i writer ma rischio la starvation dei reader

35. Albero dei parametri di allocazione e rilocalizzazione #@



36. Cosa è una primitiva di sistema e cosa comporta chiamarla #@

Una primitiva di sistema è una funzione atomica che gira ad interruzioni disabilitate con livello di privilegio sistema, viene utilizzata per l'accesso alle risorse condivise, principalmente

dispositivi di I/O, viene chiamata tramite un'interruzione software che provoca tutto il classico giro delle interruzioni (vedi domanda su architettura, è tipo la prima)

37. Cosa sono i thread #@

I thread, o processi leggeri, sono astrazioni di un programma allo stesso modo che lo è un classico processo, la differenza però è che al thread non sono associate risorse proprie, ma solamente un contesto di esecuzione, queste infatti condividono periferiche, memoria virtuale e file con altri thread dello stesso processo. Per questo il cambio di contesto di thread è molto più veloce di quello di un processo

Il multithreading permette di generare diversi thread dallo stesso processo portataggi sia in sistemi multi-cpu, in cui il vantaggio è palese, ovvero esecuzione in parallelo dei vari thread nelle cpu, sia nei sistemi monocpu o che comunque non supportano il multithreading, alleggerendo il cambio di contesto quando un thread è bloccato a causa di I/O burst

38. Condizioni di schedulabilità e fattore di utilizzazione #@

Il fattore di utilizzazione della CPU si usa principalmente con sistemi hard real-time, $U = \text{somatoria}(ci/Ti)$ dove ci = cpu_burst del processo e Ti periodo del processo.

Perché un sistema sia schedulabile (condizione **necessaria** di scheduling) $U \leq 1$ (tralasciando l'overhead) perché definendo n_i = #esecuzioni di i nel periodo T $n_a \cdot c_a + n_b \cdot c_b + n_c \cdot c_c + \dots \leq T$

La condizione **sufficiente** di schedulazione con rate monotonic richiede che $U \leq n(2^{1/n} - 1)$, che per N che tende all'infinito è uguale a 0,7

39. Sistemi hard-real time#@

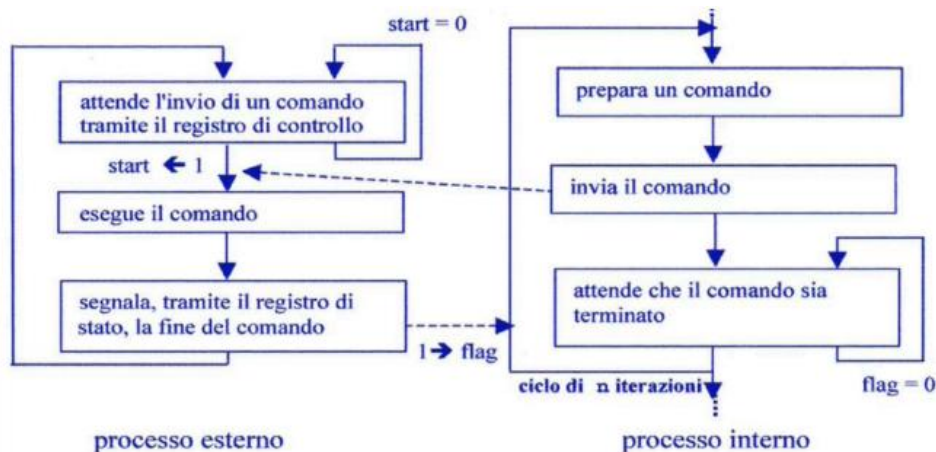
I sistemi hard real-time sono sistemi di tipo time-sharing che richiedono obbligatoriamente la conclusione dei propri processi entro la deadline, causa conseguenze catastrofiche.

I processi sono periodici entrano tutti in coda pronti al tempo t_0 e ritornano in coda ogni T_i diverso per ogni process, noi assumeremo per semplicità che la deadline corrisponda al rientro in coda pronti del processo, realisticamente è un po' più vicina, impossibile che sia più lontana.

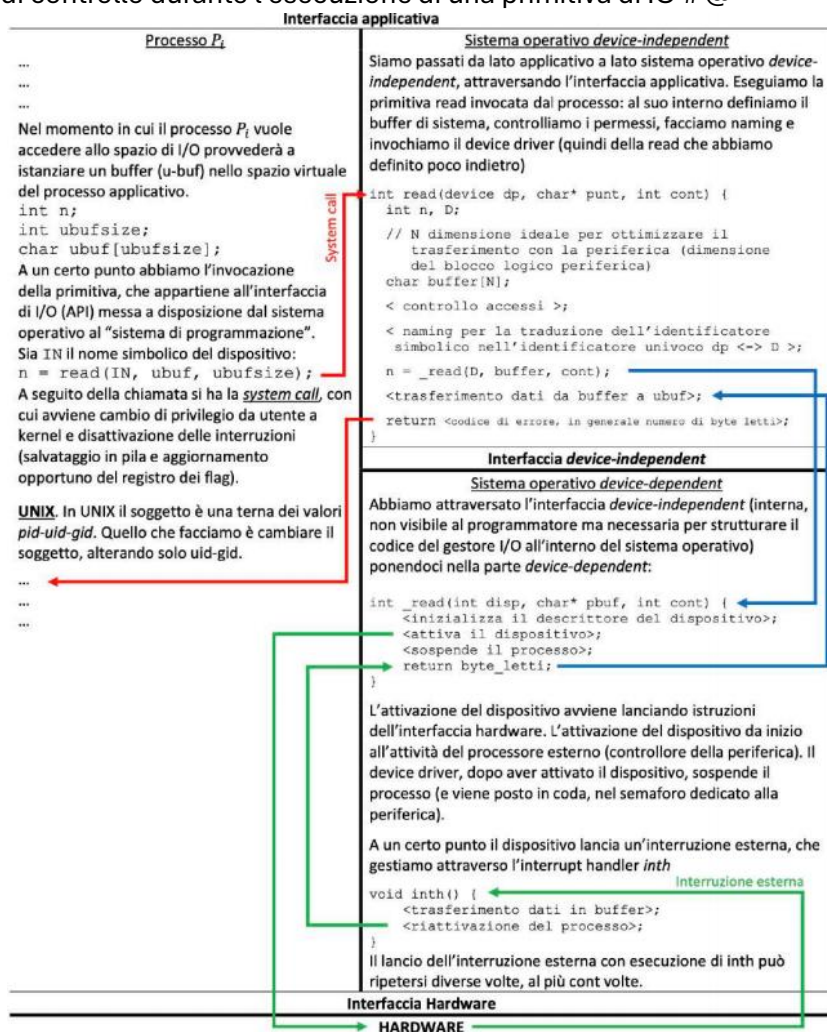
Il periodo complessivo del sistema = $\text{mcm}(T_i)$, per assicurarsi che un sistema sia schedulabile per un determinato periodo deve esserlo nel periodo complessivo.

Esiste una possibilità che questi sistemi ogni tanto debbano far girare dei processi non periodici, è importante quindi avere un po' di margine e il fattore di utilizzazione della cpu.

40. Flusso di controllo tra processo interno e processo esterno #@



41. Flusso di controllo durante l'esecuzione di una primitiva di IO # @



42. Stima del CPU_burst # @

Media esponenziale: $S_{n+1} = a t_n + (1-a) S_n$ con a apparente $[0,1]$, solitamente $= 0.5$
 t_n = durata dell'ultimo CPU_burst osservato

43. Segmentazione # @

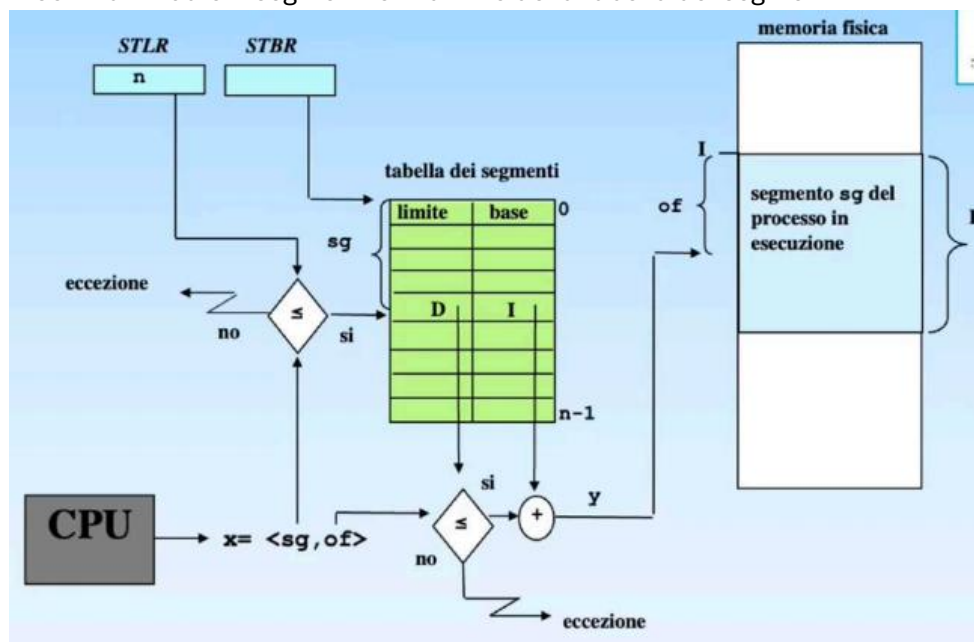
Posso dividere dati stack e codice in memoria virtuale come 3 segmenti di dimensioni diverse, nella MMU sono mantenuti i registri base e limite per ognuno di questi segmenti, per garantire la corretta rilocalazione è necessario sapere ogni indirizzo generato a quale segmento appartiene, per usare i registri delle MMU corretti.

Tutti gli indirizzi generati in fase di prelievo appartengono alla zona codice, come tutti quelli generati in fase di esecuzione di un salto. Quelli generati durante le fasi di esecuzione PUSH e POP appartengono al segmento della stack e appartengono al segmento dati tutti gli indirizzi generati nell'esecuzione delle altre istruzioni. Si genera un'eccezione se x supera il registro limite corrispondente.

La possibilità di relocare i processi in memoria consente di compattare le zone libere, diminuendo la frammentazione, dato che dopo lo `swipe_out`, grazie alla rilocalizzazione dinamica, si può modificare l'indirizzo di `swipe_in`.

Non c'è motivo però per fermarsi a 3 segmenti, è possibile segmentare il programma nei vari moduli di cui è composto: funzioni, classi, struct... in maniera da necessitare sempre di spazi contigui di dimensioni minori. I registri base e limite vengono spostati nella tabella dei segmenti.

Nel descrittore di un processo aggiungo due informazioni nel campo relativo alla memoria virtuale: #segmenti e indirizzo della tabella dei segmenti



Siccome risulta molto oneroso accedere sempre alla memoria per ottenere la traduzione di un indirizzo virtuale mantengo nel TLB un piccolo numero di traduzioni, le ultime fatte (che per i principi di località sono anche quelle più probabili a essere riferite ancora).

I segmenti sono condivisibili, ma qualora questi contengano indirizzi in memoria sono indirizzi virtuali, quindi le informazioni a cui tali indirizzi fanno riferimento devono essere allocate in posizioni identiche nello spazio virtuale dei processi.

Con la segmentazione un processo può essere interamente in memoria principale o interamente in memoria di massa, per questo si aggiungono gli stati swapped, è possibile però creare la segmentazione a domanda, che carica in memoria solo i segmenti richiesti, perde quindi subito di significato lo stato swapped, dato che un processo può essere parte di qua e parte di là. Se viene generato un indirizzo appartenente ad un segmento non in memoria principale avviene un

segmentation_fault: che comporta la copiatura del segmento dalla memoria di massa alla RAM, con ventuale rimpiazzamrnto

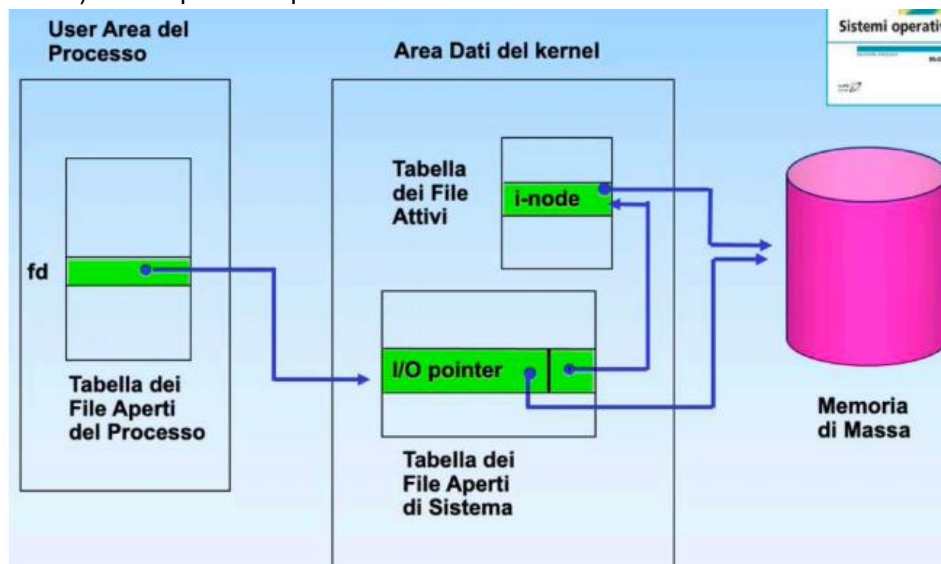
44. Cosa succede quando si apre un file(tabelle e puntatori)#@

L'accesso ad un file è subordinato all'apertura di esso, che comporta l'uso di diverse tabelle:

tabella dei file-aperti → puntata dalla user structure(parte di processo onn residente in memoria), contiene per ogni file aperto del processo un puntatore (file descriptor) alla tabella dei file aperti di sistema.

Tabella dei file aperti di sistema → presenta un entrata per ogni operazione di apertura su file(non ancora chiusi) eseguita da ogni procssso, ognuna di queste contiene un I/O pointer(ultimo record acceduto dato che accesso sequenziale) e un puntatore all' i-node del file nella tabella dei file attivi

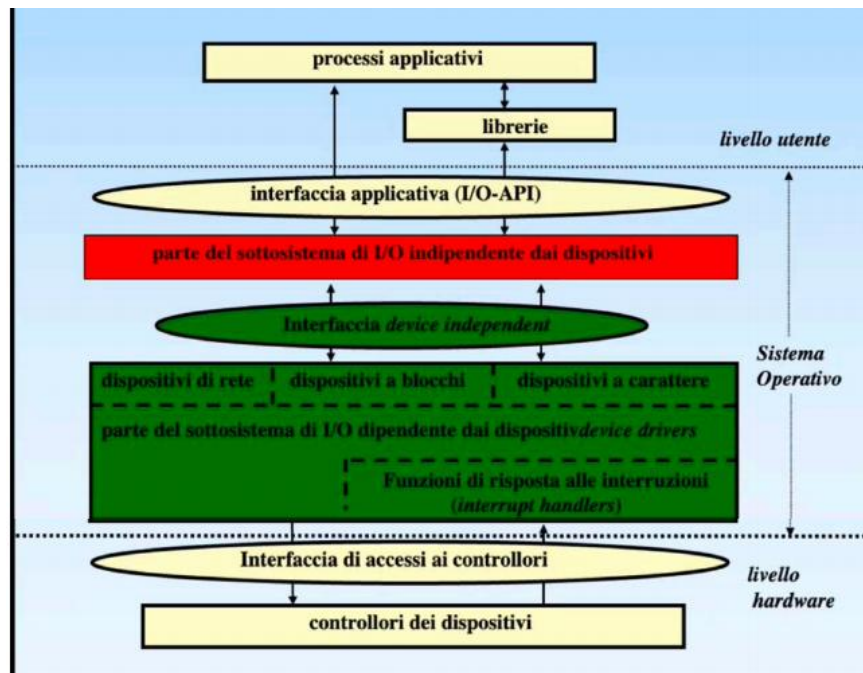
Tabella dei file attivi → contiene una copia degli i-node dei file aperti (e on ancora chiusi) da un qualsiasi processo



45. Sottosistema di I/O, livello dipendente dai dispositivi e livello indipendente dai dispositivi. #@

Il **sottosistema di I/O** si occupa di:

- definire lo spazio dei nomi per i dispositivi
- gestire i malfunzionamenti
- garantire la sincronizzazione tra P.I e P.E
- bufferrizzazione



Livello indipendente

Definisce interfaccia applicativa di I/O e fornire tutte le chiamate di sistema necessarie, dev anche interfacciarsi con la componente device dependent, per rendere più semplice l'uso dei dispositivi.

Si occupa di:

naming → consentire l'identificazione dei dispositivi mediante un nome simbolico e mettere in corrispondenza il nome col corrispondente driver.

Buffering → fornire un'area di memoria (buffer) destinata a contenere i dati durante il trasferimento, che funziona da punto di mezzo tra il dispositivo e l'indirizzo del buffer passato dall'utente. Questo porta a due vantaggi: il primo è l'elaborazione di un blocco di dati nel buffer utente in parallelo alla scrittura del dispositivo nel buffer sistema, migliorabile pure utilizzando un doppio buffer o addirittura un buffer circolare. Il secondo vantaggio invece consiste nel poter richiedere nel proprio buffer un numero di byte < alle dimensioni del blocco di trasferimento, tutto il blocco verrà passato nel buffer di sistema, ma solo il necessario in ubuf. Infine il buffer di sistema si comporta anche come una cache di blocchi nei confronti della memoria di massa.

Gestione degli errori → è buona norma gestire gli errori al livello più vicino possibile a quello in cui è stato generato, ma se il driver non riesce a risolvere l'errore il problema viene sollevato al livello superiore. Se l'errore è irrecoverable verrà propagato al livello applicativo tramite il ritorno di un codice di errore nella primitiva.

Allocazione e spooling → fornire i gestori dei dispositivi ad un processo per volta. Una tecnica spesso utilizzata è quella dello spooling. Non permettono mai al processo applicativo di avere davvero accesso alla periferica, ma ad una sua copia virtuale e privata, le operazioni che ogni processo esegue su questa copia vengono poi lette da un processo sistema (daemon) che è l'unico ad avere davvero accesso al hardware e invierà al dispositivo.

Connesso allo spooling c'è anche la questione dello scheduling delle operazioni, si potrebbe usare un algoritmo fifo o uno basato sulla priorità dei processi, dipende dal tipo di dispositivo.

Livello dipendente

Nasconde i dettagli relativi ai singoli dispositivi, ogni dispositivo viene rappresentato tramite descrittore di periferica, una classe con delle funzioni che hanno il compito di inviare gli opportuni comandi ai registri del controllore. L'insieme delle funzioni di accesso al dispositivo fisico è il device driver.

46. Come funziona il rimpiazzamento delle pagine in unix? Perché si hanno 3 variabili (des_free, lots_free, min_free) e non 2?#@

La memoria virtuale in unix utilizza la segmentazione paginata con 3 segmenti (codice, dati, stack) e l'allocazione dei segmenti viene gestita tramite caricamento delle pagine a domanda. Il processo pagedaemon si occupa del rimpiazzamento della pagina. Il kernel mantiene una descrizione dello stato di allocazione di allocazione della memoria all'interno della tabella delle pagine fisiche (core map).

Il rimpiazzamento viene fatto tramite algoritmo dell'orologio, eseguito dal pagedaemon ogni volta che il numero delle pagine fisiche libere è minore di lotsfree, può essere però che nonostante il pagedaemon la frequenza di paginazione sia troppo elevata e il numero di pagine fisiche libere rimane sempre $< \text{lotsfree}$, allora esegue lo swapper, che si occupa di fare swap-out di uno o più processi.

Vengono definite altre due variabili minfree, che indica il numero minimo di pagine fisiche libere necessarie per fare lo swap-out, e desfree, che esprime il numero di pagine fisiche libere medio desiderabile.

Lo swapper esegue solo se il numero di pagine fisiche libere $< \text{minfree}$ e il numero medio di pagine fisiche libere $< \text{desfree}$. La condizione su desfree evita che lo swapper esegua troppe volte su sistemi molto dinamici.

47. Perché esiste la tabella dei file attivi? #@
cache degli i-node

48. Scheduling per operazioni di accesso al disco #@

FCFS

La testina si muove verso il cilindro che è stato richiesto per primo

SSTF(shortest seek time first)

Dato che il movimento della testina e il movimento di rotazione del disco sono $\gg$ del tempo di operazione della cpu potrebbe essere interessante schedare le operazioni sul disco seguendo la posizione della testina, il che richiede però di ricalcolare ad ogni iterazione lo SST

SCAN

Segue il verso di movimento della testina, prima solo in un verso, poi solo nell'altro, come verso di partenza scelgo quello dello SST. Per decidere di cambiare verso devo prima scansionare tutto il vettore di richieste per verificare che non ve ne siano altre in quella direzione.

49. Matrici degli accessi in unix #@

La matrice degli accessi indica le relazioni soggetto(riga)/oggetto(colonna), in particolare i diritti che i soggetti hanno sugli oggetti e/o altri soggetti. La matrice reale è parecchio inefficiente, per cui viene implementata per righe o per colonne, nella ricerca dei diritti di un soggetto su un oggetto lavoro per righe, mentre nella modifica della tabella lavoro per colonne. La modifica dei diritti di protezione segue le regole Graham e Denning:

propagazione del diritto → è possibile passare un diritto ad altri, copiarlo o trasferirlo

assegnazione di un qualunque diritto → possibile solo se si è il proprietario

rimozione del diritto → possibile solo se si è proprietario o se si ha diritto di controllo sui soggetti.

In unix la matrice è realizzata tramite access control list(acl), ovvero per colonne, per ogni oggetto esiste una lista di coppie <oggetto, diritti del soggetto>. Quando S_j vuole fare un'operazione su O_j scorre tutta la lista di O_j in cerca della coppia con lui come soggetto.

In particolare unix divide i soggetti in owner, group owner e others.

La realizzazione per righe è detta capability list, e consiste in una lista di coppie <oggetto, permessi sull'oggetto>

Revoca e assegnamento dei diritti:

selettivi o generali? Parziale o totale? Temporanea o permanente

in ACL scorro la lista per l'oggetto e si modifica per ogni soggetto, in cl devo trovare l'oggetto in ogni lista dei soggetti

50. Come il modello bell-lapadula protegge da un trojan #@

	O_1 (Segreto)	O_2 (Pubblico)	O_3 (Pubblico)
S_1 (Segreto)	Read	Execute	Write
S_2 (Pubblico)		Execute, Owner	Read, Owner

Ipotesizzando che S₂ installi O₂ (trojan) e O₃,
 dodice propaga il suo diritto di esecuzione di O₂ a S₁ e gli assegna il diritto di scrittura di O₃
 quando S₁ eseguirà O₂ questo leggerà da O₁, e vorrebbe far scrivere il contenuto in O₃, ma
 ciò non è possibile per la priorità *

51. Chi stabilisce chi ha il diritto su cosa? #@

Le politiche di accesso agli oggetti da parte dei soggetti possono essere definite per diverse vie:

DAC(discretionary access control)

Decise dal creatore dell'oggetto

MAC(mandatory access control)

Decise da un ente centrale

RBAC(role-based access control)

diritti dipendono dal ruolo dell'utente.

È possibile costruire politiche rbac/mac su sistemi dac

Principio di privilegio minimo → contenere più possibile la distribuzione dei diritti si basa su 3 punti:

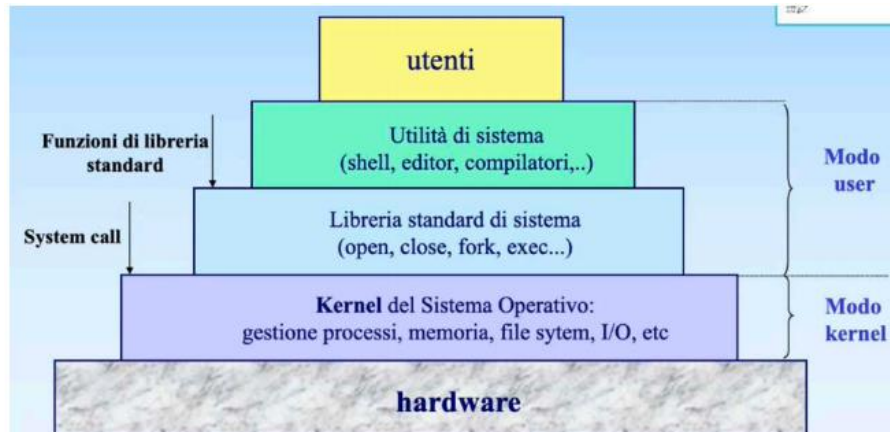
confinement → proporzionata limitazione dei diritti di accesso

sharing parameters → minimizzare dati condivisi allo stretto necessario

trojan horse → evitare che un trojan possa accedere a diversi file

52. Architettura dei sistemi unix#@

Unix nasce come sistema multiprogrammato modulare



Opera in dual mode: modo utente e modo sistema e gestisce la memoria tramite segmentazione paginata (guarda la domanda sulla gestione della memoria di unix).

Il codice dei processi è rientrante (condivisibile) grazie alla text table, memorizza i puntatori ai segmenti di codice.

Il descrittore di processo è il PCB (process control block) e si divide in:

Process structure (parte residente) contenente Pid, stato, puntatori ai segmenti, info sullo scheduling, Pid del padre, info sui segnali, puntatore ad un altro process structure e puntatore alla user structure.

User structure (non residente) contiene il contesto, informazioni sulle risorse allocate, informazione sui segnali, ambiente del processo.

Ogni processo (anche i processi leggeri) ha oltre allo stack utente anche uno stack nel kernel

53. File system di unix #@

Il file system è rappresentabile come un albero, lo stesso file può avere più path (nomi simbolici) ma un solo i-node (descrittore) contenuto nella i-list (lista dei descrittori) all'indice i-number.

Il descrittore del file contiene i seguenti campi:

Tipo: ordinario, directory, speciale (es. dispositivi → → in unix tutto è un file)

Owner e group owner

Dimensioni: # record, ogni record = 1 byte

Data di creazione e modifica

12 bit di protezione

#link simbolici

Vettore di 13/15 indici di blocchi per accesso al file in memoria di massa.

Blocco = 512 byte → un blocco contiene 128 indirizzi(indirizzi da 32 bit)

La dimensione massima di un file = 1GB + 8MB + 64 KB + 5KB

Indirizzione tripla | doppia | singola | direttamente

File di piccole dimensioni sono allocati in modo contiguo

Allocazione
mista

Il disco virtuale è diviso in 4 sezioni:

boot block: estensione delle ROM, utilizzato in bootstrap

Super block: definisce queste 4 sezioni e mantiene un puntatore alla lista dei blocchi liberi ed uno alla lista degli i-number nella i-list

i-list

Data block

SCRITTO

1. Comandi di base e metacaratteri

Comandi base della shell testuale

cd → naviga tra le cartelle

pwd → stampa path assoluto della directory corrente

ls → stampa il contenuto della cartella indicata

-l → mostra dettagli dei file -a → mostra anche file nascosti

man → apre il manuale dell'istruzione indicata

whatis → breve descrizione dell'istruzione

apropos → ricerca una parola nelle pagine del manuale

mkdir/rmdir → crea/elimina(se vuota) una cartella

cp src dst → copia un file in un altro o in una directory

cp/mv src1 src2 ... dst → copia muove più file o directory in una directory

mv src dst → rinomina una file o una directory

touch → crea un file/ aggiorna il timestamp

cat → concatena il contenuto di 1 o più file e lo stampa in stdout

less → visualizza un file poco per volta

head/tail -n → mostra le prime/ultime n righe di un file(n default = 10)

echo → stampa nello stdout

su → cambia utente nel terminale

sudo → esegue un'istruzione come root

> → invia lo stdout di un comando ad un file(sovrascrive)

>> → invia stdout (append) 2> → invia stderr &> → invia entrambi

< → recupera input da un file

| → collega output di un comando all'input del successivo

Metacaratteri

* → sostituisce uno o più caratteri

? → sostituisce un carattere

[a,h] → sostituisce un carattere con uno tra questi

[a-h] → sostituisce un carattere nell'insieme da a a h

2. Cosa si trova in /etc/passwd, /etc/shadow, /etc/group e /etc/gshadow @ #

passwd

Contiene info pubbliche sugli utenti, visualizzabile con `sudo vipw`

Username:x:uid:gid:altre info:/home/utente:/bin/shell

x = password cifrata presente

shadow

contiene le password cifrate degli utenti, visualizzabile con `sudo vipw -s`

username\$algoritmo di hashing\$salt\$hash(salt+password):ultima modifica: eta

minima: eta massima: tempo di avviso: periodo di inattività

algoritmo identificato da un intero

salt è una costante per complicare l'hash

eta minima e massima è riferita ovviamente alla password

tempo di avviso prima che scada la password

group

contiene info pubbliche sui gruppi, visualizzabile con `sudo vigr`

gruppo:x:gid:lista utenti

gshadow

contiene le password cifrate dei gruppi + lista amministratori, visibile con `sudo vigr -s`

gruppo:password:gid:amministratori:membri

3. Quali sono e cosa contengono le sottocartelle principali della directory principale #@

/ → directory principale

/etc → file di configurazione

/media → supporti rimovibili

/sbin → programmi di sistemi

/home → home directory degli utenti

4. spiega a cosa servono e come si settano SUID e SGID e come si vedono. #@

i bit SUID e SGID servono per permettere al processo che esegue un determinato file di farlo utilizzando i privilegi dell'owner o del group owner.

Se sono settati, con il comando `ls -l` si potrà vedere al posto del diritto di esecuzione "x" group owner e/o del owner "s"

Per settarli/rimuoverli si può usare sia la rappresentazione simbolica che quella ottale:

rappresentazione simbolica: `sudo chmod o+s file`, `sudo chmod g-s file`,

rappresentazione ottale `sudo chmod 2777 file`, `sudo chmod 4777 file`, `sudo chmod 6777 file`, `sudo chmod 777 file` (2 =SGID, 4=SUID, 6=entrambi)

5. Come un utente può modificare la priorità di un processo #@

Un utente può modificare la priorità di un processo agendo sulla sua niceness, un valore numerico[-20,19], che è inversamente proporzionale alla priorità.

`nice -n n comando` → esegue un processo con niceness = n

`renice n pid` → modifica la nice del processo pid ad n

solo root può modificare la niceness ad un valore < 0 e/o minore del valore precedente

6. Descrizione e differenze di find e locate #@

L'istruzione di find è molto più precisa e permette una ricerca più raffinata grazie ai test ei connettori logici, permette anche di eseguire determinate istruzioni sui file trovati

Test

-name 'pattern' → riconosce un pattern nel nome del file

-type [d,f,l] → il file è di tipo directory, file o link simbolico

-size [+,-]n [c,k,M,G] → la dimensione è maggiore o minore di n B,KB, MB, GB

-user 'uid/username' -group 'gid/groupname' → riconosce owner e group-owner

-perm [-,/] mode → richiede almeno/ almeno uno dei permessi indicati in mod, siano essi in rappresentazione ottale o simbolica.

Azioni

Delete → elimina i file trovati

Exec command {} \; → specifica un comando da eseguire {} viene sostituito dal nome del file trovato.

L'istruzione locate invece risulta più semplice da usare: locate file. Ricerca il file in tutto il filesystem sfruttando un database aggiornato dal sistema mediante sudo update db, il db non si aggiorna in tempo reale, per cui potrebbe causare confusione. Inoltre spesso l'istruzione locate deve essere installata, mentre la find è già presente.

7. Significato dei bit di protezione per file e cartelle #@

I bit di protezione sono rwx/s: nei file read permette di leggerne il contenuto, mentre nelle directory permette di vedere i file e le sottocartelle al suo interno. Il bit w permette di scrivere in un file e di modificare il contenuto di una directory. Il bit x permette di eseguire un file o di accedere alla cartella. Infine il bit s permettere di utilizzare i privilegi di owner e group owner.

8. GREP #@

L'istruzione grep recupera da uno più file le righe che corrispondono ad un determinato pattern: grep [-e] 'espressione' [-e 'espressione'] file... la prima -e è necessaria se uso più espressioni.

[ab] → sceglie un carattere tra a e b [a-b] → sceglie un carattere compreso

^stringa → l'espressione deve trovarsi in testa

Stringa\$ → l'espressione deve trovarsi in coda

. → rappresenta un qualsiasi carattere

* → l'espressione che lo precede può essere ripetuta 0 o più volte

9. Come funziona il pilotaggio di un'applicazione #@

È possibile pilotare un'applicazione tramite l'ausilio delle funzioni pipe e dup2.

Int **pipe**(int fd[2]) è una funzione che prende in ingresso due descrittori di file ed inserisce in fd[0] l'estremo di lettura della pipe, e in fd[1] l'estremo di scrittura, restituisce 0 se tutto ok, -1 altrimenti. È molto utile per la comunicazione padre figlio dato che i figli ereditano i descrittori di file dal padre. È importante per ogni processo chiudere l'estremo che non usa per evitare deadlock e comportamenti inattesi.

Read sull'estremità di lettura risulta bloccante solo se la pipe è vuota e c'è almeno uno scrittore attivo, altrimenti restituisce 0 il messaggio o EOF.

Write sull'estremità di scrittura è bloccante solo se il buffer è pieno.

La funzione `int dup2(int target, int new fd)` copia il descrittore `target` in `newfd`, se il file indicizzato da `newfd` in precedenza era aperto viene chiuso prima, restituisce un valore < 0 in caso di errore

`Dup2(pipe[1], stdout_fileno)` → tutto ciò che veniva stampato dallo `std_out` ora verrà rediretto verso l'estremità di scrittura del file.

Per pilotare un'applicazione posso:

- 1) Creo due pipe per una comunicazione bidirezionale e chiudo gli estremi non utilizzati
- 2) `Fork()`
- 3) Redirigo `std_in` e `std_out` del figlio nelle due pipe
- 4) Trasformo il figlio nell'applicazione tramite `exec`

Esempio di pilotaggio di applicazione che riceve un valore di input e stampa il quadrato, il quadrato del quadrato e così via

```
#include<unistd.h>
#include<stdlib.h>
#include<stdio.h>
#include<string.h>

int main(int argc, char* argv[]) {
    int FtS[2]; // father 2 son
    int StF[2]; // son 2 father

    pipe(FtS);
    pipe(StF);

    pid_t pid;

    pid = fork();

    if(pid == 0){//figlio

        // chiudo le estremità non utilizzate
        close(FtS[1]);
        close(StF[0]);

        // duplico FtS[0] e lo metto al posto di stdin
        dup2(FtS[0],STDIN_FILENO);
        close(FtS[0]);

        // duplico StF[1] e lo metto al posto di stdout
        dup2(StF[1],STDOUT_FILENO);
        close(StF[1]);

        // eseguo l'applicazione
        //( per semplicità ipotizzo che sia nella stessa directory dell'eseguibile)
        execl("applicazione","applicazione",NULL);
    }
```

```

}
else{ //padre
    int nread = -1;
    int i = 0;
    char sendbuffer[1024];
    char receivebuffer[1024];
    long unsigned int numero= argc>1?atoi(argv[1]):2;

    close(FtS[0]);
    close(StF[1]);

    nread=read(StF[0],receivebuffer,sizeof(receivebuffer)-1);
    receivebuffer[nread] = '\0';
    printf("Ricevuto:%s\n",receivebuffer);

    while(i<4){

        snprintf(sendbuffer,sizeof(sendbuffer),"%lu\n",numero);
        write(FtS[1],sendbuffer,strlen(sendbuffer));

        nread=read(StF[0],receivebuffer,100);
        receivebuffer[nread] = '\0';
        printf("Ricevuto:%s\n",receivebuffer);

        numero = atoi(receivebuffer);
        i++;
    }
}
}

```

10. TAR #@

Istruzione per creazione di un archivio, compressione ed estazione di file:

tar modalità [opzioni][lista di file]

modalità

a → aggiunge un tar file all' archivio c → crea archivio
 --delete → cancella file da archivio r → aggiunge file ad archivio
 t → elenca file dall' archivio x → estrae dall' archivio
 u → aggiunge file all'archivio solo se non già presente

opzioni

v → modalità verbose z → comprime con gzip → .tar.gz
 j → comprime con bzip → tar.bz2 → più veloce ma più grande
 f → specifica nome dell' archivio

11. Sintassi e comportasmento di open,read,write, close #@

Int **open**(const char*path, int flags)

Ritorna il file descriptor e prende in ingresso il path del file(relativo o assulo) e chiede che tipo di accesso si vuole fare →MACRO: O_APPEND, O_RDONLY, O_WRONLY, O_RDWR

Int **close**(int fd)

Chiude il file indicizzato da fd e ritorna 0 se ok, -1 se errore

Ssize_t **read**(int fd, void *buf, size_t count)

Prende in ingresso il file descriptor, l'indirizzo del puntatore al buffer dove si vogliono scrivere i dati letti e il numero di byte da leggere, restituisce il numero di byte letti.

Ssize_t **write**(int fd, const void* buff, size_t count)

Prende in ingresso il file descriptor, indirizzo del buffer da cui leggere i dati da scrivere e il numero di byte da scrivere, restituisce il numero di byte scritti.

12. Cosa è un segnale e quali sono gli effetti della ricezione di un segnale su un processo e come si ridefinisce un handler#@

Un segnale in ingresso è una notifica di un evento asincrona dal processo A al processo B, alla ricezione di un segnale questo viene ignorato oppure viene eseguito un handler, che può essere di default o ridefinito dall'utente. Esistono diversi segnali, i più importanti sono:

SIGHUP → il terminale è stato chiuso

SIGINT → forza l'interruzione del processo(ctrl +c)

SIGKILL → interruzione immediata, non ignorabile

SIGUSR1/SIGUSR2 → liberi per la ridefinizione dell'utente, di default terminano il processo

SIGALARM → timer scaduto

SIGSTOP → sospende temporaneamente il processo in attesa di SIGCONT(ctrl + z)

È possibile ridefinire un handler con una qualsiasi funzione di tipo void con almeno un parametro intero(valore del segnale) e assegnarla all'arrivo del segnale tramite la primitiva `signal_t signal(int sig, signal_t handler)`.

Se handler vale SIG_IGN il segnale viene semplicemente ignorato, se vale SIG_DFL ripristina handler di default. La primitiva ritorna l'indirizzo del precedente handler.

Un eventuale handler sovrascritto viene ereditato da un figlio (se non c'è ancora stata la fork), una ulteriore sovrascrittura dell'handler del figlio non modificherà quello del padre.

Se il figlio si trasforma tramite exec mantiene solo i segnali da ignorare

13. Cosa è il job control e quali sono i comandi #@

Il job control permette di sospendere/riattivare gruppi di processi. La shell associa un `job_id` ad ogni comando eseguito, salvati in una tabella richiamabile dal comando **jobs**. Se il programma lanciato contiene delle `fork()`, i figli faranno parte dello stesso process group, e quindi dello stesso job. Dato che i jobs sono un concetto della shell e non del kernel questi non vengono riconosciuti da altri terminali.

I jobs si dividono tra quello che esegue in foreground, che controlla il terminale(`std_in, std_out, std_err`), e quelli in background, è possibile spostare un job in

foreground o background tramite **fg/bg job_id**, oppure posso lanciare un job direttamente in background con **comando &**. Si può sospendere il job in foreground tramite sigstep(**ctrl + z**).

Kill %job_id invia sigterm al job indicato.

Cosa succede se il terminale viene chiuso?

Se si chiude il terminale i processi del job in esecuzione ricevono tutti un segnale di sighup, che di norma li termina, è possibile non far terminare i processi con una delle due istruzioni:

nohup comando → il job eseguito è immune a sighup ma non ha più accesso allo standard input e l'output è rediretto a nohup.out

disown %job_id → il job non riceverà sighup, bisogna manualmente occuparsi del non permettere di leggere lo standard input e ridirezionare lo standard output.

14. Descrivi tutte le librerie necessarie studiate #@

stdlib.h → libreria standard per molte funzioni e definizioni di uso comune in c

stdio.h → libreria per le funzioni di I/O scanf e printf

unistd.h → libreria che definisce le funzioni per i processi

sys/wait.h → definisce le macro per la wait WIFEXITED(status) e WEXITSTATUS(status)

signal.h → definisce le macro per i segnali (e forse anche le funzioni)

pthread.h → definisce tutte le primitive dei thread

string.h → definisce alcune funzioni di utilità per le stringhe

15. Thread #@

i thread, chiamati anche processi leggeri, sono un insieme di flussi di esecuzione indipendenti dal processo. Un codice è thread safe se evita interazioni non volute e le risorse condivise vengono accedute in mutua esclusione.

Ogni thread è identificato da un tid di tipo pthread_t, che è un tipo opaco, il primo tid può essere recuperato tramite `gettid()` o `pthread_self()`, posso paragonare due tid tramite la `pthread_equal(tid1, tid2)`.

per creare un thread esiste la primitiva `int pthread_create(pthread_t*thread, const pthread_attr_t* A, void*(*start routine)(void *), void *arg)`.

la funzione restituisce 0 se tutto ok, altro altrimenti. Prende in ingresso un puntatore al tid creato, attributi del thread (default = NULL), puntatore alla funzione che gira nel thread, argomento della funzione codice del thread.

Un thread può terminare tramite `void pthread_exit(void*retval)`, ed in questo modo i figli continueranno ad eseguire, ed i `retval` verrà salvato il valore di ritorno del thread. Se il thread termina senza `exit` i figli vengono terminati.

La `void pthread_join(pthread_t thread, void **retval)` sospende il thread in attesa che un altro termini, prende in ingresso il tid del thread di cui si aspetta la terminazione e l'indirizzo del puntatore di `retval` della `exit`. Restituisce 0 se funziona, codice di errore altrimenti (altrimenti = esiste già join sullo stesso tid rischio deadlock)

16. Mutua esclusione nei thread #@

La mutua esclusione nei thread è realizzata tramite semaforo binario mutex per proteggere l'accesso alle risorse condivise, un semaforo viene dichiarato "pthread_mutex_t m;"; dopodiché inizializzato tramite **pthread_mutex_init**(pthread_mutex_t * m, const pthread_mutexattr_t* attr), che ha in ingresso il puntatore al mutex da inizializzare e il puntatore alla struttura dati con attributi di inizializzazione (default = NULL). È ovviamente possibile fare lock e unlock come se fossero wait e signal, queste due operazioni ritornano poi 0 in caso di successo, errore altrimenti.

```
int pthread_mutex_lock(pthread_mutex_t m)
```

```
int pthread_mutex_unlock(pthread_mutex_t m)
```

17. Sincronizzazione nei thread e le condition variables #@

Le condition variables sono utilizzate dai thread per la sincronizzazione diretta in attesa di una condizione apposita.

```
Pthread_cond_t c; //dichiarazione
```

```
int pthread_cond_init(pthread_cond_t* c, pthread_cond_attr_t* attr) → analogo della mutex_init
```

Sulle condition_variables sono definite solo le operazioni di wait e signal/broadcast.

La sospensione dei thread sulle variables condition deve essere del tipo

```
pthread_mutex_lock(m);  
while(condizione) //quando il thread si sospende si libera il mutex associato  
    pthread_cond_wait(c, m); //viene rifatto il lock quando il thread  
pthread_mutex_unlock(m); //si sveglia, se occupato si sospende sul mutex
```

il mutex serve per la condizione, dato che sarà un controllo su una variabile condivisa.

Inoltre si usa un while e non un if perché quando il thread viene svegliato non è detto che vada subito in esecuzione lui, per cui potrebbe andare prima in esecuzione un altro thread che impedisce a questo di superare la condizione

```
int pthread_cond_signal/broadcast(pthread_cond_t * c) sveglia uno a caso o tutti i processi bloccati nella variable condition. Deve essere sempre invocata dentro la sezione critica, così siamo sicuri che mentre viene invocata la condizione è rispettata.
```

18. Differenza hard-link/soft-link#@

Hard e soft link possono riferire tutti lo stesso i-node, ma l'esistenza di un i-node è legata esclusivamente agli hard-link. Un file può essere riferito da n hard link e m soft link, poi posso iniziare ad eliminare uno a uno tutti i soft-link e hard link. L'i-node continuerà ad esistere solo fintanto che esso è riferito da almeno un hard-link. Se elimino l'ultimo hard-link l'i-node viene eliminato prescindendo dal numero di soft-link che lo riferiscono

19. Chmod

- Cosa permette di fare il comando chmod?
- Descrivere la rappresentazione simbolica e ottale
- Spiegare cosa si visualizza per quanto riguarda i permessi del file fileName al termine dei comandi
 - chmod ugo=rwx fileName
 - chmod g-x fileName

iii. `ls -l fileName`

il comando `chmod` permette di modificare i permessi di lettura, scrittura e accesso ai file e directory (che comunque sono sempre trattati come file) e può essere eseguito solo da root o dall'owner

`chmod [ugo] [+,-,=] [rwx/s] [-r] file` → rappresentazione simbolica: è possibile scegliere uno o più tra i soggetti di riferimento (owner, group owner, others), un simbolo per assegnazione, rimozione o aggiunta e uno o più diritti. `-r` serve solo ad applicare i diritti anche a tutti i file dentro la cartella (se è una cartella)

`chmod [2,4,6] abc file` → rappresentazione ottale: a b c rappresentano numeri in base 8, 1 = esecuzione, 2 = scrittura, 4 = lettura, a seconda dei diritti che si desidera inserire si fa la somma e si ottiene a (per owner) b (group owner) e c (others). Il primo numero 2 = SUID, 4 = SGID, 6 = ENTRAMBI

i → assegno tutti i diritti a tutti

ii → al group owner elimino il diritto di esecuzione
verrà visualizzato `rwrx-xrwx`

20. Spiegare il seguente codice#

```
int main() {
    int count = 0;
    int i = 0;

    while (i < 2) {
        int pid = fork();
        if (pid > 0) {
            wait(NULL);
        }
        else {
            count++;
        }
        i++;
    }
    fprintf(stdout, "count = %d", count);
}
```

In questo codice inizialmente il padre si blocca e prosegue solo il figlio, poi anche il figlio si blocca e prosegue figlio ... insomma gira che ti rigira stampa 2 1 1 0

21. Descrivere le primitive `fork()`, `exit()`, `wait()`, `exec()` cosa succede ai processi, valori di ritorno. #@
Queste primitive sono definite nella libreria `unistd.h`.

fork() genera un processo figlio e ritorna al padre l'id del figlio e 0 al figlio o un valore negativo se la fork è fallita.

exit(int status) termina volontariamente un processo e inserisce lo stato di terminazione nella variabile `status`.

Wait(int * status) sospende il processo padre in attesa della terminazione di un figlio, e salva nella variabile `status` lo stato di terminazione del figlio. Nel registr `<sys/wait.h>` sono definite due macro `WIFEXITE(status)` che ritorna vero se il figlio è terminato volontariamente e `WEXITSTATUS(status)` che ritorna il parametro della `exit`.

EXEC() invece è una famiglia di istruzioni, che sostituisce il codice del processo. In particolare la funzione **execl**("char * path", "char *arg0", ..., NULL) trasforma il codice del processo in un comando da terminale. Path è il path del comando, arg0 il nome del comando arg1, argn le possibili opzioni e altri argomenti, NULL il terminatore. Se ha successo on ritorna nulla, altrimenti esegue il codice successivo a questa primitiva.

22. Descrivere le primitive signal(), kill(), pause(), alarm(), sleep()#@

Signal() → vedi domanda sugli handler

Int Kill(pid_t pid, int sig) → invia al processo pid il segnale sig. e ritorna 0 se ha successo

pid > 0 al processo specifico

pid = 0 a tutti i processi del suo stesso process group

pid == -1 a tutti a cui è possibile

pid < -1 a tutti i processi con process group = -pid

int pause() → sospende il processo in attesa di un segnale, ritorna -1 se l'handler non termina l'esecuzione del processo

unsigned int alarm(unsigned int seconds) → setta un timer alla fine del quale il processo riceve un segnale di sigalarm(default termina il processo). Se ne vengono fatte due o più l'ultima sovrascrive le precedenti. Ritorna 0 se è il primo o i secondi mancanti alla scadenza dell'allarme precedente se non lo è.

Unsigned int sleep(unsigned int seconds) → setta un allarme e poi va in sleeping, il processo si risveglierà quando riceverà un qualsiasi segnale non ignorato

23. Cosa sono argc e argv[]:#@

quando si esegue un programma da linea di comando è possibile inserire gli argomenti del main direttamente da li, è necessario esplicitare come parametri formali del main int argc e *argv[], che corrispondono rispettivamente al numero di paramteri attuali passati da linea di comando e un array di stringhe i parametri stessi. Il primo è sempre il nome del programma eseguito (es a.out = argv[0]);

24. Esercizio accesso ad una risorsa condivisa senza superare un numero massimo di thread.

25. 26. 27. Sono gli esercizi dell'esercitazione 9